# LAPORAN UJIAN TENGAH SEMESTER

# PEMOGRAMAN APLIKASI BERBASIS WEBSITE

Diajukan untuk memenuhi persyaratan kelulusan Mata Kuliah

Pemograman Aplikasi Berbasis Website

Oleh :

**Fiqri Mochamad Fadilah**

**247006111051**

**PROGRAM STUDI INFORMATIKA**

**FAKULTAS TENIK**

**UNIVERSITAS SILIWANGI**

**2025**

# KATA PENGANTAR

Puji syukur dipanjatkan ke hadirat Tuhan Yang Maha Esa atas terselesaikannya laporan praktikum mata kuliah Pemrograman Aplikasi Berbasis Website mengenai pengembangan sistem "Enjambre Game Store". Laporan ini disusun sebagai bentuk dokumentasi teknis dan analisis fungsional terhadap implementasi perangkat lunak berbasis web yang telah dikembangkan.

Pengembangan proyek ini menitikberatkan pada integrasi arsitektur *frontend* modern, mencakup penggunaan *Asynchronous JavaScript* melalui Fetch API untuk komunikasi data dengan *server-side*. Fokus utama praktikum diarahkan pada pemahaman alur data dinamis, mulai dari proses otentikasi pengguna hingga manajemen status aplikasi (*state management*) menggunakan *Web Storage API*. Implementasi *LocalStorage* menjadi poin krusial dalam memastikan persistensi data pada fitur keranjang belanja dan koleksi *library* agar tetap terjaga meski terjadi siklus muat ulang halaman (*page reload*).

Proses identifikasi dan penanganan kendala teknis (*debugging*) menjadi bagian signifikan dalam penyusunan laporan ini. Analisis terhadap beberapa kegagalan eksekusi skrip, seperti error *undefined property* pada pemformatan mata uang dan sinkronisasi *scope* fungsi pada logika *checkout*, telah dilakukan untuk memastikan stabilitas sistem. Evaluasi objektif terhadap kode program dilakukan guna menghasilkan aplikasi yang tidak hanya memiliki antarmuka yang presisi, tetapi juga logika program yang efisien.

Analisis yang tersaji dalam laporan ini mencakup seluruh siklus fungsionalitas aplikasi, mulai dari antarmuka galeri produk, mekanisme distribusi data ke keranjang, hingga validasi akhir pada tahap simulasi transaksi. Diharapkan laporan ini dapat berfungsi sebagai referensi teknis yang memadai dalam memahami fundamental pengembangan aplikasi web yang bersifat *stateful* dan interaktif.

Disadari bahwa masih terdapat ruang untuk optimalisasi lebih lanjut dalam sistem ini. Oleh karena itu, segala bentuk kritik dan saran teknis yang membangun sangat diharapkan untuk pengembangan kompetensi di masa mendatang.

Tasikmalaya, 10 Maret 2025

# DAFTAR ISI

KATA PENGANTAR ..... i
DAFTAR GAMBAR ..... iii
BAB I ..... 1
I.1. Latar Belakang ..... 1
I.2. Lingkup ..... 2
I.3. Tujuan ..... 2
BAB II ..... 4
II.1. Alat dan Persiapan yang Dilakukan ..... 4
II.2. Pengenalan lingkungan kerja ..... 4
II.3. Langkah-Langkah Melakukan Praktikum ..... 5
II.4. Hasil yang Didapat ..... 20
BAB III ..... 28
III.I. Kesimpulan Praktikum ..... 28
III.II. Analisis Dan Evaluasi ..... 28
DAFTAR PUSTAKA ..... 30

# DAFTAR GAMBAR

Gambar 1: Tampilan Website ..... 1
Gambar 2:Tampilan Cart ..... 3
Gambar 3:Pembuatan Folder Projek ..... 5
Gambar 4:Tampilan VS Code ..... 6
Gambar 5:tampilan instalasi Live Server ..... 7
Gambar 6:Tampilan Instalasi Thunder Client ..... 7
Gambar 7:Tampilan Folder Awal ..... 8
Gambar 8:File-File Developer ..... 8
Gambar 9:Index.html ..... 9
Gambar 10:Main CSS ..... 10
Gambar 11:Link ke JS ..... 10
Gambar 12:Data Game ..... 11
Gambar 13:Tampilan Terminal VS Code ..... 11
Gambar 14:Mengaktifkan Server Lokal ..... 12
Gambar 15: IP Lokal ..... 12
Gambar 16:Kode Pembuatan Log In ..... 12
Gambar 17:Implementasi Event Listener pada Submit Login ..... 13
Gambar 18:Aktivasi Live Server ..... 14
Gambar 19:Tampilan Log In Awal ..... 14
Gambar 20:Konfigurasi Request POST untuk Autentikasi ..... 15
Gambar 21:Penanganan Response dan Ekstraksi Token ..... 15
Gambar 22:Penyimpanan Token pada LocalStorage ..... 16
Gambar 23:Implementasi Header Authorization Bearer ..... 16
Gambar 24:Pengambilan Data Produk dengan Request GET ..... 17
Gambar 25:Penyusunan Logika Pembuatan Akun ..... 17
Gambar 26:Kode Show Key ..... 17
Gambar 27: Tampilan Log In ..... 18
Gambar 28:Tampilan Show Key(Hanya dalam simulasi) ..... 18
Gambar 29:Tampilan Salah/Error ..... 19
Gambar 30:Kode Manajemen Session Nama Pengguna ..... 19
Gambar 31:Fungsi Validasi Login Global ..... 20
Gambar 32:Tampilan Dashboard ..... 21
Gambar 33:Tampilan Perenderan Produk Dinamis dan Filter ..... 21
Gambar 34:Tampilan Fungsionalitas icon keranjang ..... 22
Gambar 35:Tampilan data bertambah ke cart.html ..... 23
Gambar 36:Tampilan Cart.html ..... 24
Gambar 37:Tampilan Game Bertambah" ..... 24
Gambar 38:Tampilan Checkout.html ..... 24
Gambar 39: Tampilan Library sebelum Checkout ..... 25
Gambar 40:Notifikasi Pembelian Berhasil ..... 25
Gambar 41:Tampilan Game Bertambah ..... 25
Gambar 42:Server dimatikan/Stop ..... 26
Gambar 43:Tampilan Dashboard Jika Server Lokal Mati ..... 26
Gambar 44:Tampilan Library Saat Server Lokal Mati ..... 27

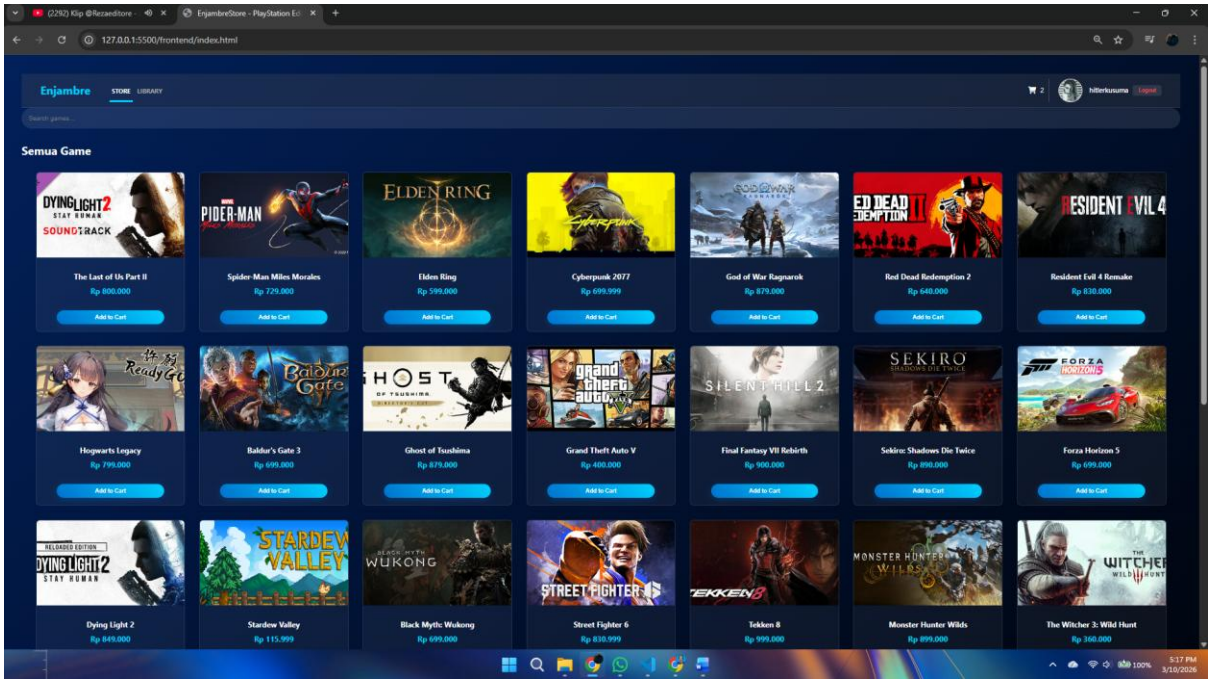
# BAB I

# PENDAHULUAN

## I.1. Latar Belakang

Perkembangan teknologi informasi telah mengubah paradigma distribusi perangkat lunak dari media fisik menjadi platform distribusi digital yang bersifat dinamis. Dalam ekosistem pengembangan web modern, sebuah toko digital tidak hanya berfungsi sebagai katalog statis, melainkan sebagai aplikasi yang mampu mengelola status data pengguna secara *real-time*. Implementasi sistem *E-Commerce* seperti "Enjambre Game Store" menjadi relevan untuk dipelajari karena mencakup fundamental pemrograman web yang krusial, yaitu integrasi antara antarmuka pengguna (UI) dengan logika penyimpanan data di sisi klien (*client-side storage*).

Masalah utama dalam pengembangan aplikasi web seringkali terletak pada persistensi data bagaimana memastikan informasi seperti keranjang belanja tidak hilang saat navigasi halaman terjadi. Penggunaan *Web Storage API* (LocalStorage) menjadi solusi teknis yang efisien untuk menangani masalah tersebut tanpa ketergantungan penuh pada *database* server di setiap aksi pengguna. Melalui praktikum ini, dilakukan analisis mendalam mengenai bagaimana JavaScript mengelola objek data JSON untuk mensimulasikan alur transaksi profesional, mulai dari validasi stok, perhitungan harga otomatis, hingga pemindahan hak akses produk ke dalam *Library* pengguna.




*Gambar 1: Tampilan Website*

## I.2. Lingkup

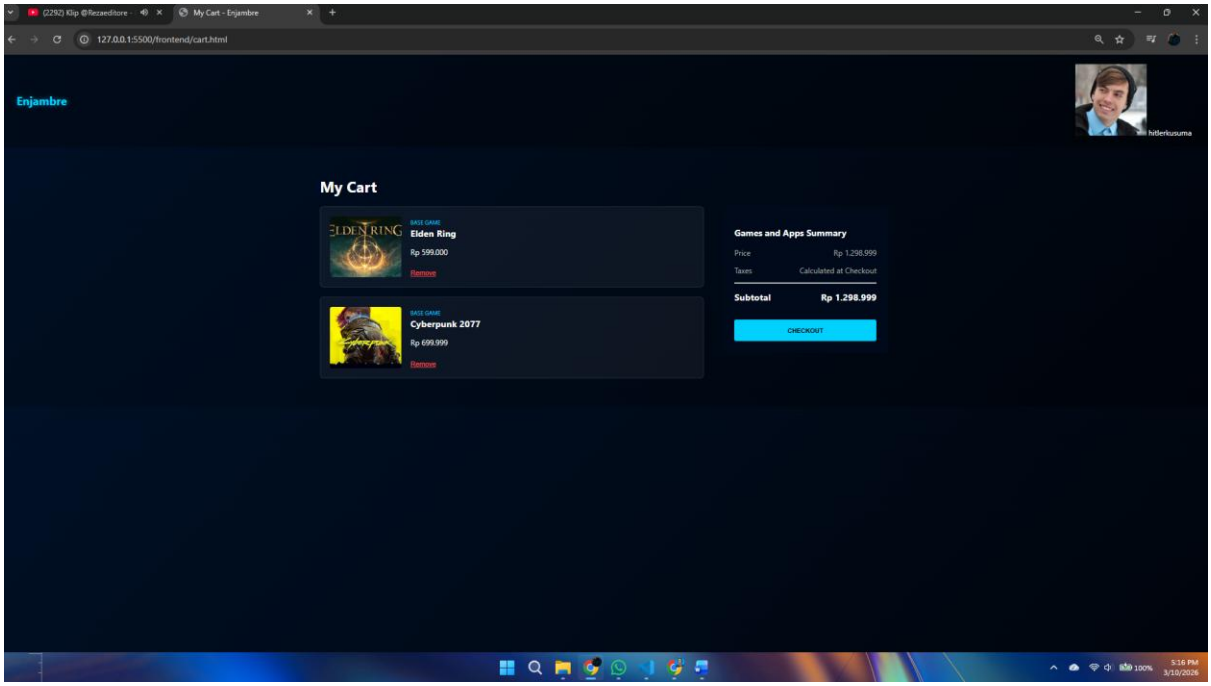
Lingkup praktikum ini dibatasi pada pengembangan *frontend* dan logika bisnis di sisi klien untuk aplikasi "Enjambre Game Store". Komponen-komponen utama yang menjadi cakupan analisis meliputi:

- Penerapan desain responsif menggunakan CSS Grid dan Flexbox untuk layout toko, keranjang, dan koleksi.
- Mekanisme pengambilan data produk melalui Fetch API dan penanganannya menjadi elemen DOM dinamis.
- Logika manajemen keranjang belanja (*Shopping Cart*) yang mencakup fungsi penambahan, penghapusan, dan kalkulasi total harga.
- Simulasi alur transaksi (*Checkout*) yang memvalidasi ketersediaan data sebelum dilakukan perpindahan status kepemilikan produk.
- Pengelolaan persistensi data koleksi game pada halaman *Library* yang bersifat permanen di level peramban (*browser*).

## I.3. Tujuan

Tujuan dari pelaksanaan praktikum dan penyusunan laporan ini adalah:

1. Menganalisis efektivitas penggunaan *Vanilla JavaScript* dalam mengelola manipulasi DOM secara dinamis berdasarkan data dari API.
2. Mengimplementasikan *LocalStorage* sebagai instrumen penyimpanan data sementara untuk menjaga status aplikasi (*State Management*) antarhalaman.
3. Membedah logika sinkronisasi data pada fitur keranjang belanja untuk menghasilkan perhitungan nilai transaksi yang akurat.
4. Mengevaluasi penanganan error pada skrip JavaScript, khususnya terkait properti *undefined* saat pengolahan data JSON yang dapat mengganggu fungsionalitas aplikasi.
5. Menghasilkan purwarupa (*prototype*) platform distribusi game yang memiliki alur operasional logis dari tahap pemilihan hingga kepemilikan produk.




*Gambar 2:Tampilan Cart*

# BAB II

# PRAKTIKUM

## II.1. Alat dan Persiapan yang Dilakukan

Persiapan praktikum melibatkan penyediaan perangkat keras dan perangkat lunak yang mendukung pengembangan aplikasi web berbasis *client-side scripting*. Alat-alat yang digunakan adalah sebagai berikut:

1. **Perangkat Keras (*Hardware*)**: Unit Laptop Asus Tuf F16 dengan spesifikasi standar yang mampu menjalankan peramban modern dan penyunting kode secara simultan.
2. **Penyunting Kode (*Code Editor*)**: Visual Studio Code digunakan sebagai lingkungan pengembangan utama karena dukungannya terhadap ekstensi *IntelliSense* untuk JavaScript dan integrasi terminal.
3. **Peramban Web (*Web Browser*)**: Google Chrome digunakan sebagai media eksekusi dan pengujian aplikasi. Fitur *Chrome Developer Tools* (Inspect Element) digunakan secara intensif untuk memantau *Console Log*, jaringan API, dan status *LocalStorage*.
4. **Server Lokal**: Ekstensi *Live Server* pada VS Code digunakan untuk mensimulasikan protokol HTTP secara lokal, guna menghindari masalah kebijakan *Cross-Origin Resource Sharing* (CORS) saat memuat modul JavaScript.
5. **Aset Digital**: Gambar sampul game (format .jpg/.png) dan data JSON yang bersumber dari *endpoint* API distribusi game.

## II.2. Pengenalan lingkungan kerja

Lingkungan kerja pengembangan "Enjambre Game Store" dirancang secara modular untuk memisahkan antara struktur antarmuka, gaya visual, dan logika operasional. Berikut adalah rincian arsitektur berkas yang digunakan:

- **Sistem Navigasi File**: Seluruh berkas diatur dalam satu direktori proyek utama dengan pembagian peran yang spesifik. Berkas .html berfungsi sebagai kerangka dasar, berkas .css untuk manajemen *layouting* dan tema gelap (*dark theme*), serta berkas .js untuk penanganan logika dinamis.
- **Ruang Lingkup DOM (*Document Object Model*)**: Lingkungan kerja memanfaatkan elemen-elemen HTML sebagai target manipulasi JavaScript. Sebagai contoh, elemen dengan ID cart-items-list diatur sebagai wadah dinamis yang akan diisi oleh skrip saat data keranjang dimuat.
-

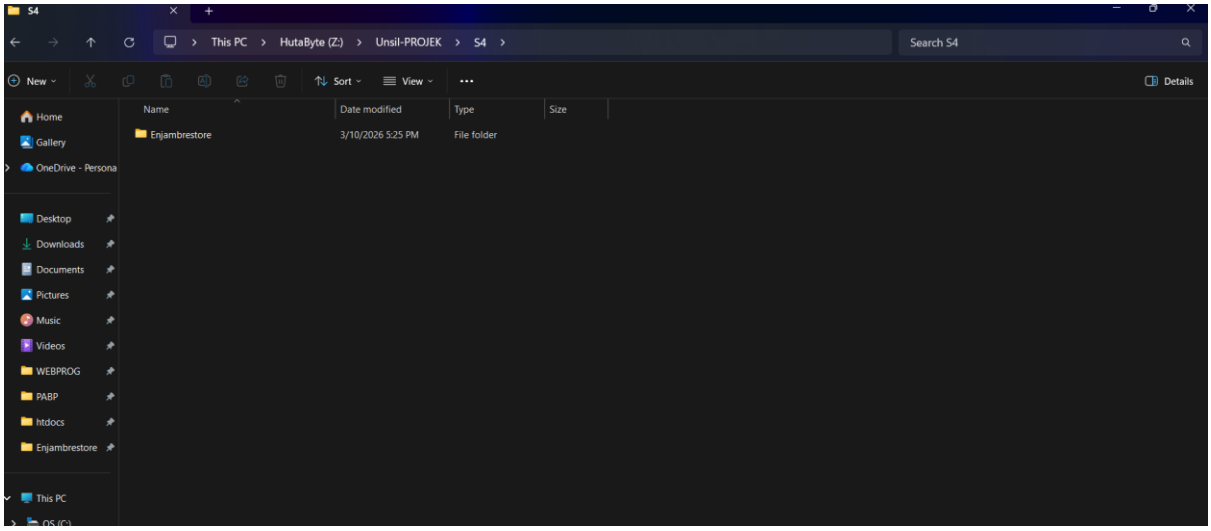
- **Mekanisme Penyimpanan Lokal (*Storage Environment*)**: Lingkungan kerja ini menggunakan fitur *Application Storage* pada peramban. Data keranjang belanja disimpan di bawah kunci enjambre_cart, sementara data koleksi disimpan pada kunci enjambre_library.
- **Konsol Pengembang (*Developer Console*)**: Merupakan bagian vital dari lingkungan kerja untuk mengidentifikasi kesalahan sintaksis atau kegagalan pemanggilan properti. Analisis pada konsol membantu dalam mendeteksi *TypeError* saat variabel tertentu memiliki nilai *undefined* sebelum fungsi toLocaleString() dipanggil.

## II.3. Langkah-Langkah Melakukan Praktikum

Proses pengembangan "Enjambre Game Store" dilakukan melalui tahapan sistematis untuk memastikan integrasi data yang akurat dan pengalaman pengguna yang logis. Berikut adalah urutan langkah-langkah yang dilakukan:

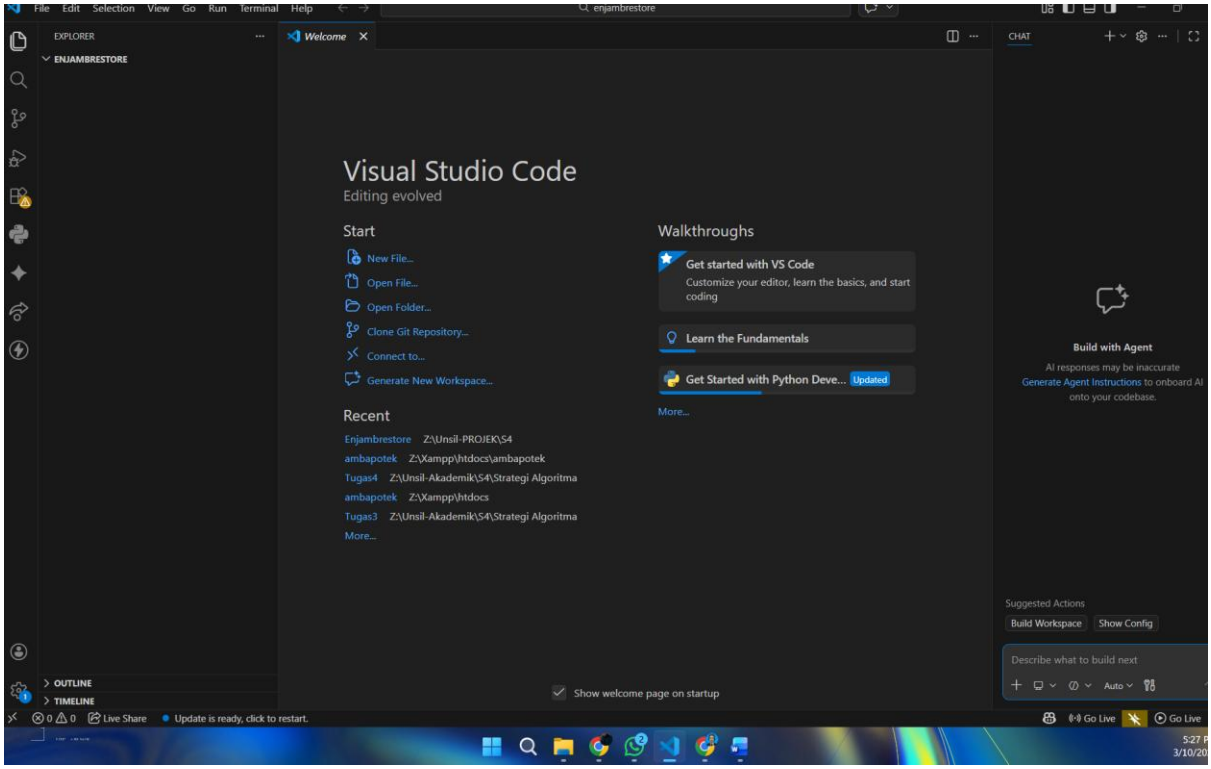
**Fase 1: Konfigurasi Environment dan Inisialisasi Proyek**

Inisialisasi Direktori Kerja Utama Langkah awal dimulai dengan pembuatan folder induk proyek bernama Enjambre-Store melalui terminal atau file explorer. Hal ini bertujuan untuk memusatkan seluruh aset dan kode sumber dalam satu *root directory* agar referensi *pathing* file bersifat relatif dan konsisten.



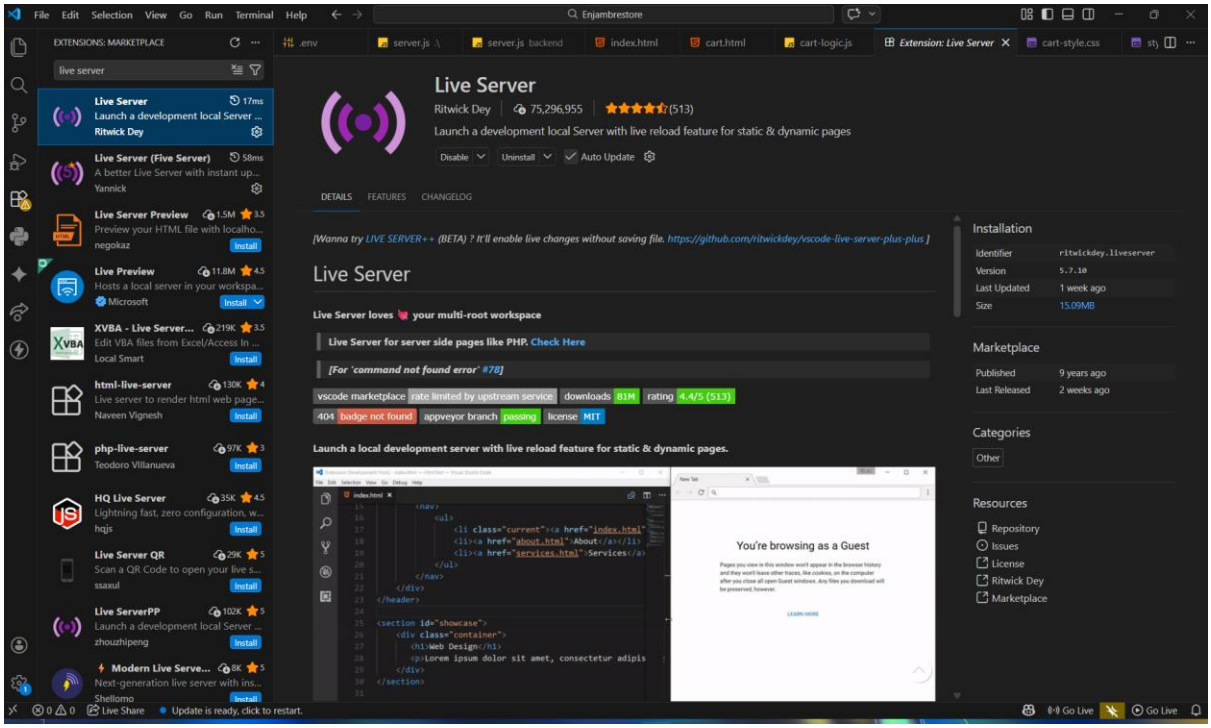

*Gambar 3:Pembuatan Folder Projek*

Pembukaan Proyek pada Visual Studio Code Folder yang telah dibuat dibuka menggunakan fitur "Open Folder" pada Visual Studio Code. Langkah ini mengaktifkan lingkungan kerja (*workspace*) dan memungkinkan penggunaan fitur-fitur IntelliSense untuk manajemen file yang lebih efisien.

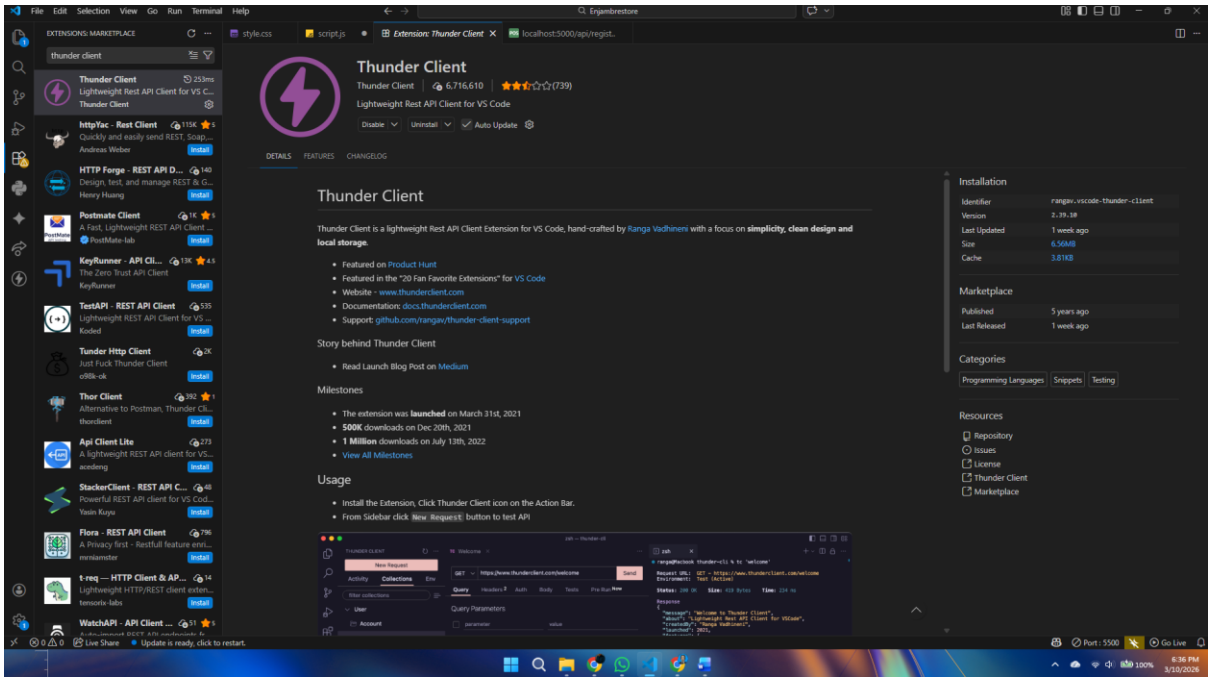



*Gambar 4:Tampilan VS Code*

Instalasi Ekstensi "Live Server" Melalui menu *Extensions* (Ctrl+Shift+X), dilakukan instalasi ekstensi "Live Server" oleh Ritwick Dey. Alat ini sangat krusial dalam pemrograman web untuk menjalankan server lokal yang mendukung *Auto-Reload*, sehingga setiap perubahan kode pada HTML atau JavaScript akan langsung terefleksi di peramban tanpa *refresh* manual.

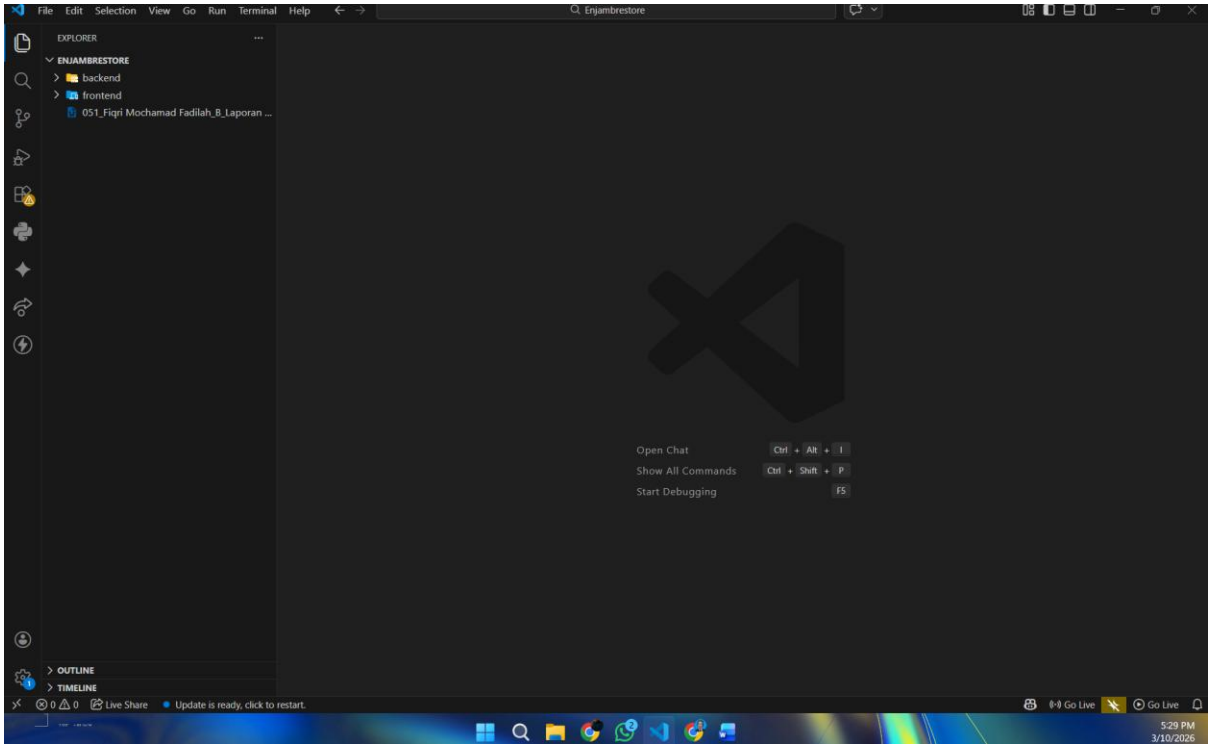



*Gambar 5:tampilan instalasi Live Server*

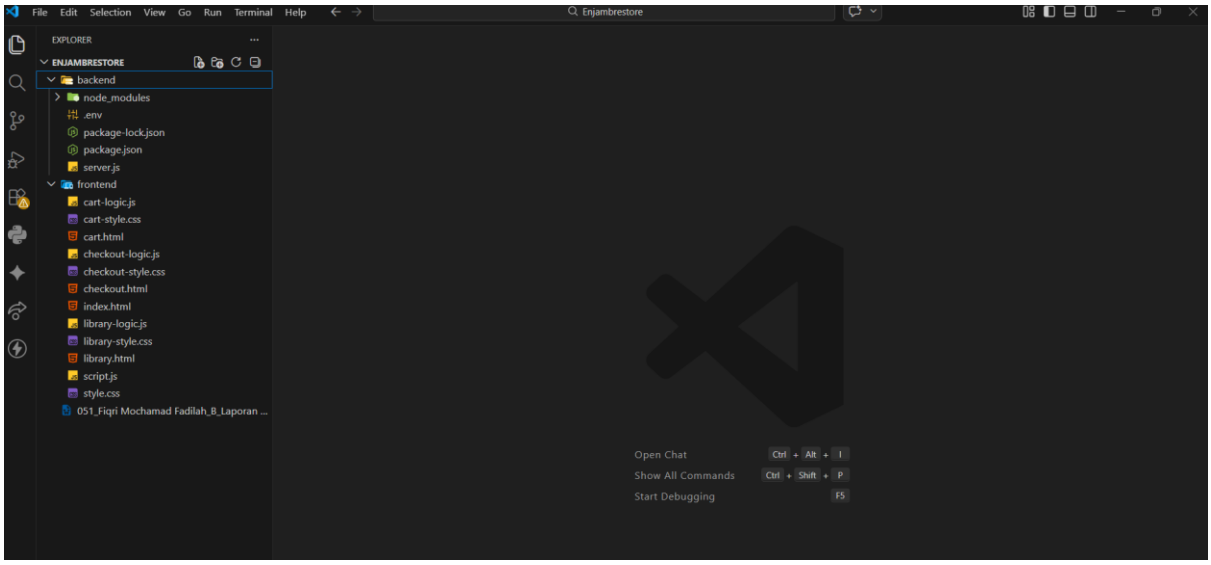



*Gambar 6:Tampilan Instalasi Thunder Client*

Pembuatan Struktur Folder Modular Di dalam VS Code, dibuat sub-folder khusus seperti /css untuk menyimpan file *stylesheet*, /js untuk logika program, dan /assets untuk menyimpan gambar produk game. Pemisahan ini dilakukan untuk mematuhi prinsip *Separation of Concerns* dalam pengembangan perangkat lunak.

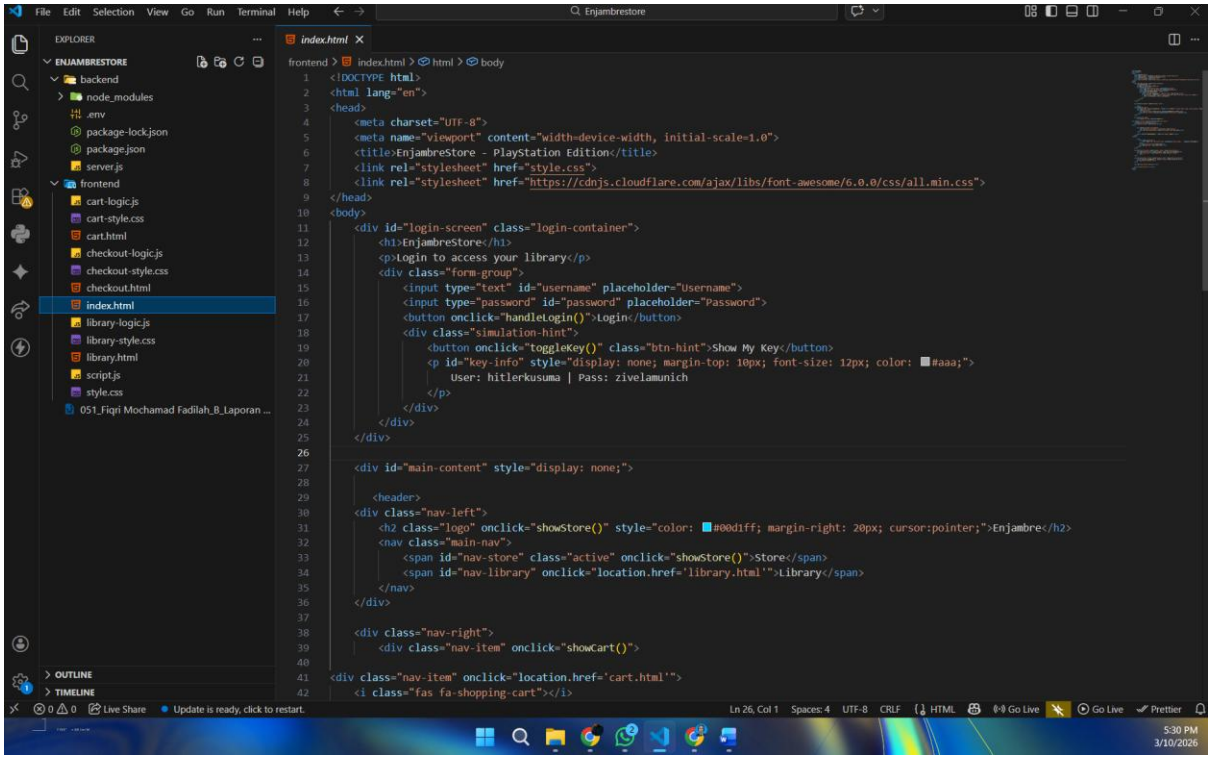



*Gambar 7:Tampilan Folder Awal*



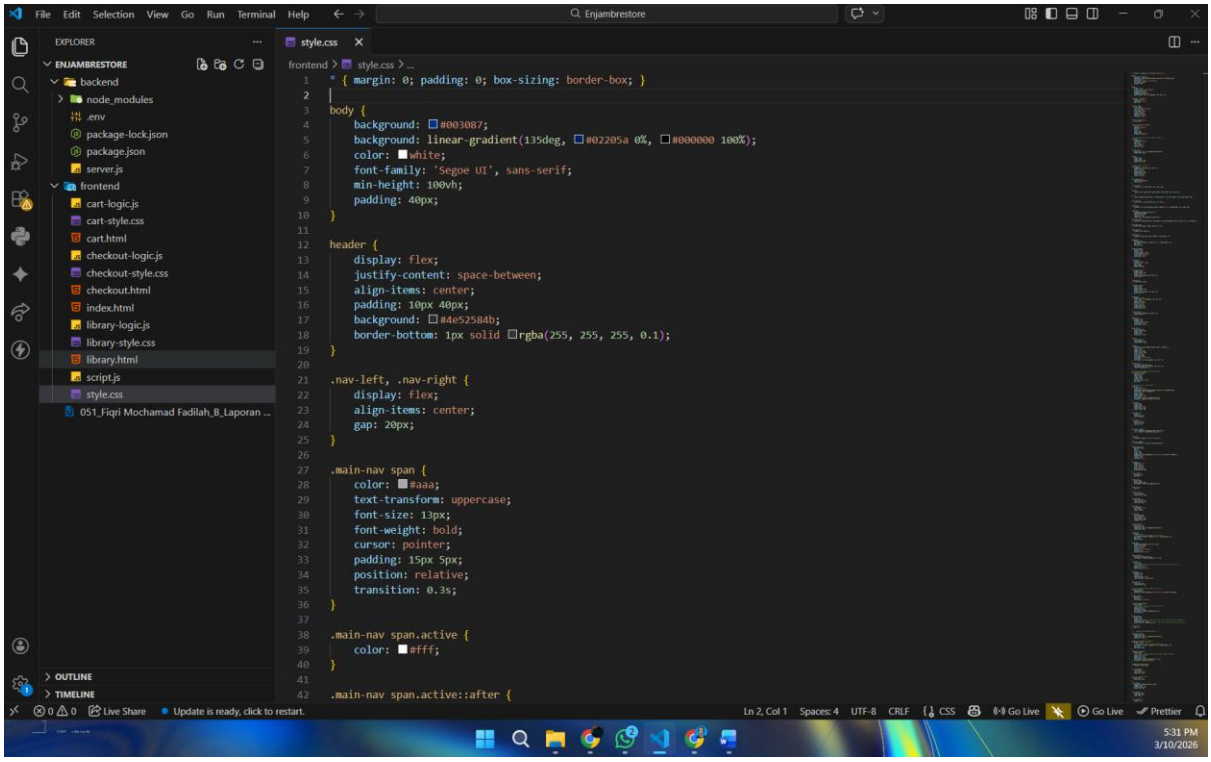

*Gambar 8:File-File Developer*

Pembuatan File Keranjang (index.html) Dibuat berkas index.html sebagai titik masuk utama aplikasi. Penggunaan *shortcut* emmet (!) diterapkan untuk menghasilkan struktur dasar dokumen HTML5 secara cepat, termasuk elemen <head>, <body>, dan metadata responsivitas viewport.



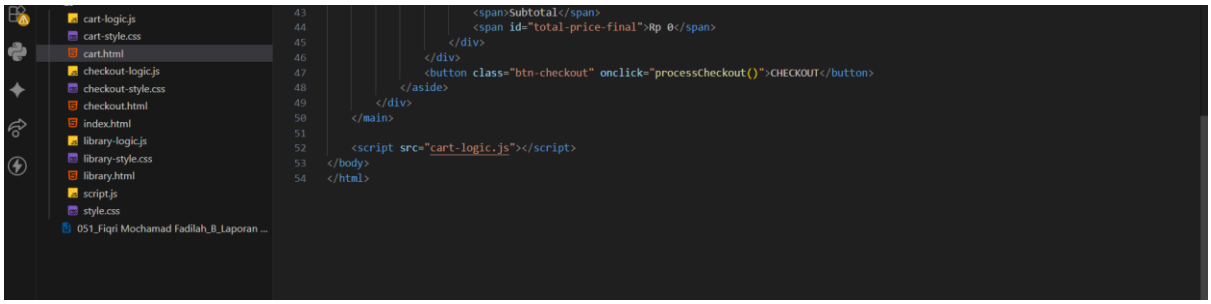

*Gambar 9:Index.html*

Konfigurasi CSS Reset dan Font Global Dibuat berkas style.css di dalam folder /css. Langkah ini melibatkan penghapusan *margin* dan *padding* bawaan peramban (*CSS Reset*) serta pemilihan font (seperti Segoe UI atau Montserrat) untuk memastikan tampilan konsisten di berbagai perangkat.




*Gambar 10:Main CSS*

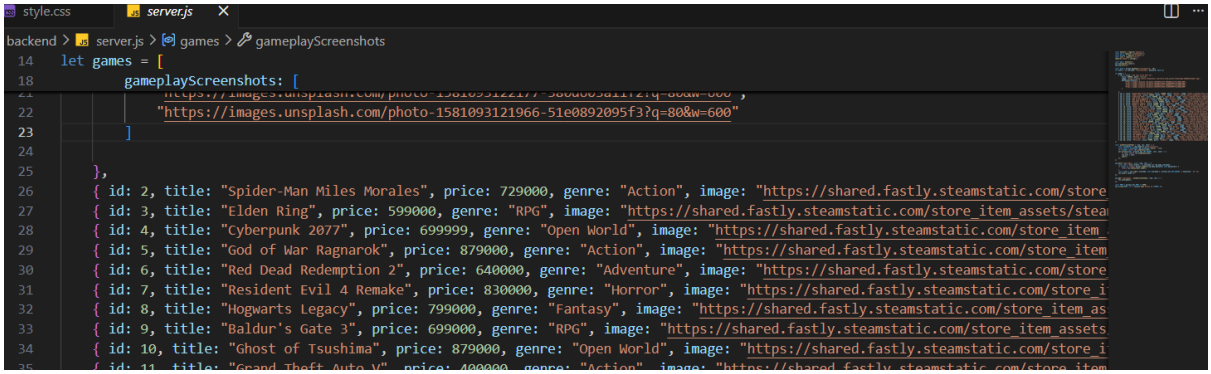
Inisialisasi File Logika Utama (app.js) Dibuat berkas app.js di dalam folder /js. Berkas ini dihubungkan ke index.html menggunakan tag <script src="js/app.js" defer></script>. Atribut defer dipastikan ada agar skrip dieksekusi setelah seluruh elemen DOM selesai dimuat.




*Gambar 11:Link ke JS*

```
<script src="script.js"></script>
```

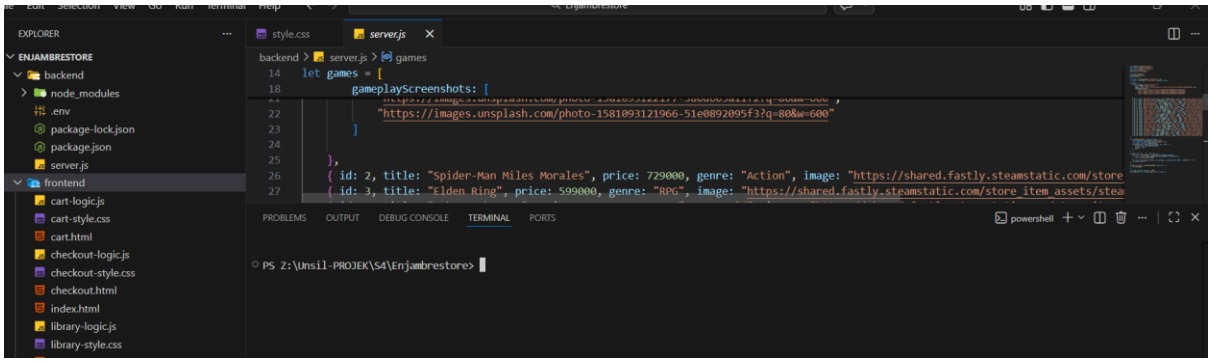
Penyiapan Aset Gambar Produk Gambar-gambar sampul game (GTA V, Cyberpunk, dll.) dimasukkan ke dalam folder /assets. Nama file disederhanakan (tanpa spasi) untuk menghindari error pemanggilan URL pada atribut src elemen gambar nantinya.




*Gambar 12:Data Game*

```
{ id: 2, title: "Spider-Man Miles Morales", price: 729000, genre: "Action",
image:
"https://shared.fastly.steamstatic.com/store_item_assets/steam/apps/1817190/he
ader.jpg" },
```

Konfigurasi Terminal Terintegrasi Terminal internal VS Code dibuka (Ctrl+`) untuk memantau status eksekusi atau menjalankan perintah Git di masa mendatang. Penggunaan terminal ini memudahkan proses *monitoring* tanpa harus berpindah jendela aplikasi.

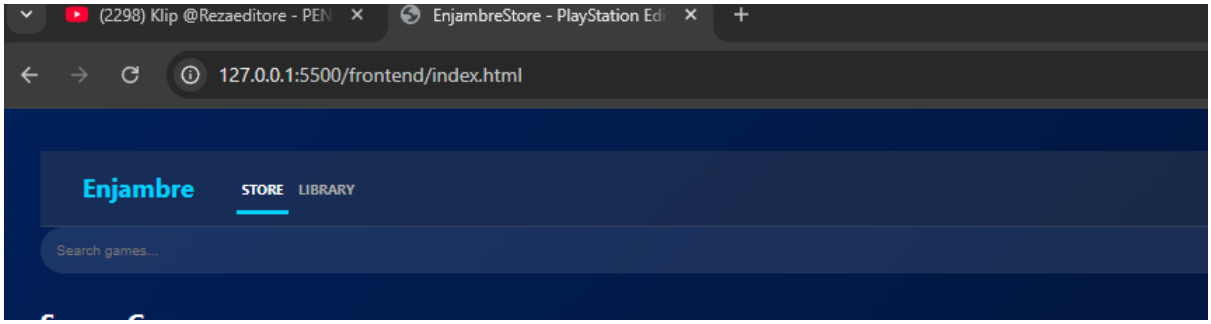



*Gambar 13:Tampilan Terminal VS Code*

Aktivasi Live Server dan Pengujian Koneksi Klik kanan pada index.html dan pilih "Open with Live Server". Browser akan terbuka otomatis pada alamat http://127.0.0.1:5500. Langkah ini menandai bahwa *environment* pengembangan telah siap sepenuhnya untuk tahap pengkodean fungsional.

```
backend > server.js > games
14  let games = [
18          gameplayScreenshots: [
22              "https://images.unsplash.com/photo-1581093121966-51e0892095f3?q=80&w=600"
23          ]
24
25      },
26      { id: 2, title: "Spider-Man Miles Morales", price: 729000, genre: "Action", image: "https://shared.fastly.s
27      { id: 3, title: "Elden Ring", price: 599000, genre: "RPG", image: "https://shared.fastly.steamstatic.com/st

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS Z:\Unsil-PROJEK\S4\Enjambrestore> cd backend
PS Z:\Unsil-PROJEK\S4\Enjambrestore\backend> ^C
PS Z:\Unsil-PROJEK\S4\Enjambrestore\backend> node server.js
[dotenv@17.3.1] injecting env (2) from .env -- tip: write to custom object with { processEnv: myObject }
Active on 5000
```

*Gambar 14:Mengaktifkan Server Lokal*




*Gambar 15: IP Lokal*

## Pembuatan Antarmuka Form Login

Penyusunan elemen <form> pada file HTML untuk menangkap input *username* dan *password* dilakukan dengan atribut required. Struktur ini dirancang untuk memastikan validasi awal di sisi klien sebelum data dikirim ke server melalui skrip JavaScript.

```
</head>
<body>
    <div id="login-screen" class="login-container">
        <h1>EnjambreStore</h1>
        <p>Login to access your library</p>
        <div class="form-group">
            <input type="text" id="username" placeholder="Username">
            <input type="password" id="password" placeholder="Password">
            <button onclick="handleLogin()">Login</button>
            <div class="simulation-hint">
                <button onclick="toggleKey()" class="btn-hint">Show My Key</button>
                <p id="key-info" style="display: none; margin-top: 10px; font-size: 12px; color: #aaa;">
                    User: hitlerkusuma | Pass: zivelamunich
                </p>
            </div>
        </div>
    </div>

    <div id="main-content" style="display: none;">
```

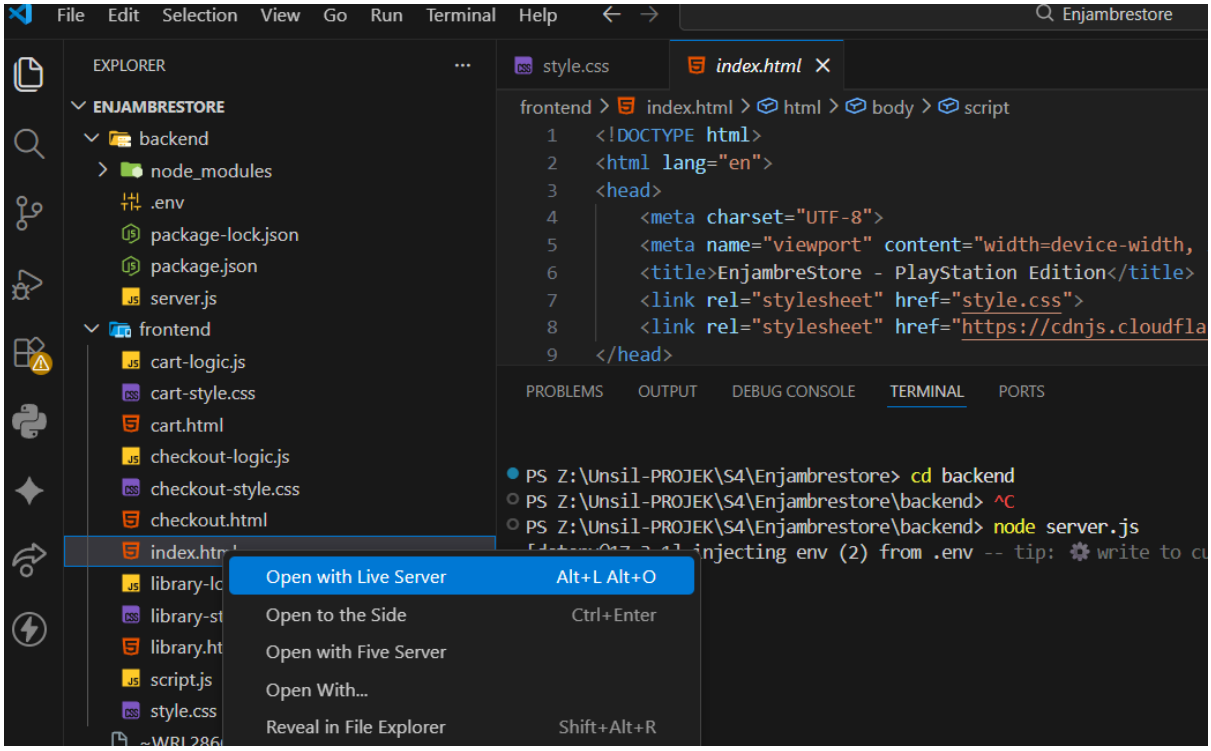
*Gambar 16:Kode Pembuatan  Log In*

### Implementasi Event Listener pada Submit Login

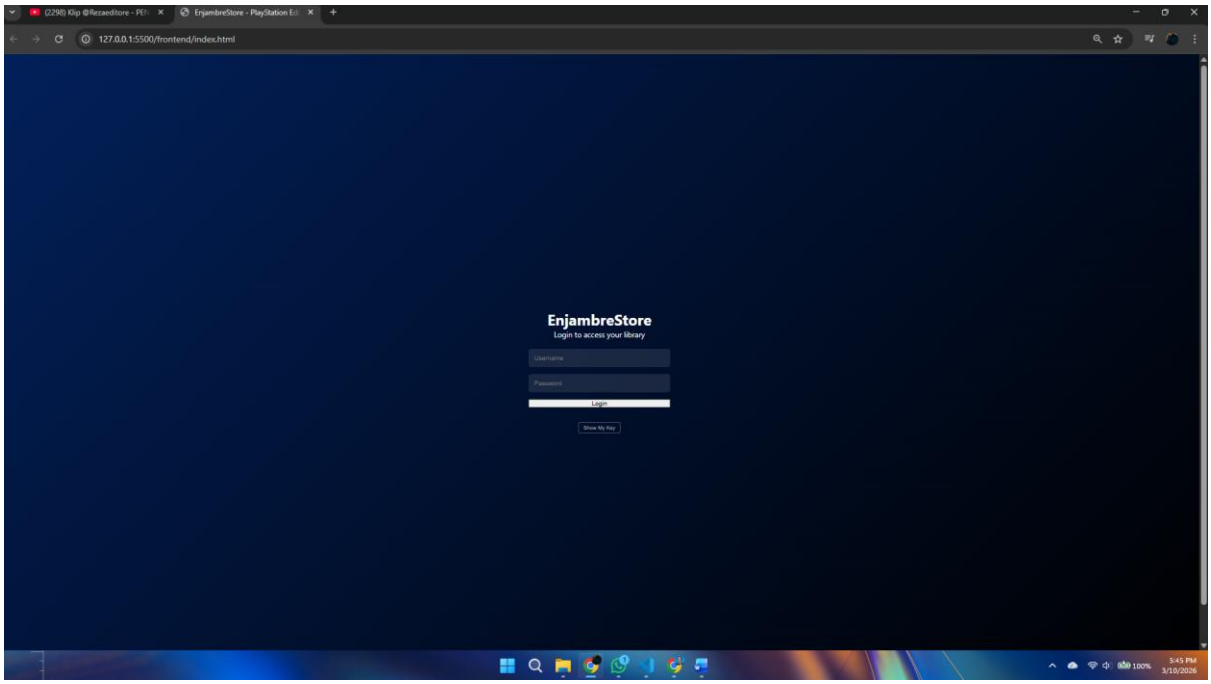
Fungsi JavaScript dihubungkan ke form login menggunakan metode addEventListener('submit'). Penggunaan e.preventDefault() diaplikasikan untuk mencegah perilaku bawaan browser yang melakukan *reload* halaman, sehingga proses autentikasi dapat dikelola secara sepenuhnya melalui latar belakang (*asynchronous*).

```
tend > script.js > ...

let STORE_DATA = [];
let ACCESS_TOKEN = localStorage.getItem('enjambre_token') || "";
let USERNAME = localStorage.getItem('enjambre_user') || "";

document.addEventListener('DOMContentLoaded', () => {
    const isLoggedIn = localStorage.getItem('isLoggedIn');

    if (isLoggedIn === 'true' && ACCESS_TOKEN) {

        showMainContent();
    } else {

        showLoginScreen();
    }
    updateCartBadge();
});

async function handleLogin() {
    const userVal = document.getElementById('username').value;
    const passVal = document.getElementById('password').value;

    if (!userVal || !passVal) return alert("Isi username dan password!");

    try {
        const response = await fetch('http://localhost:5000/api/login', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ username: userVal, password: passVal })
        });

        if (response.ok) {
            const result = await response.json();

            // Simpan ke LocalStorage agar tidak hilang saat refresh
            localStorage.setItem('isLoggedIn', 'true');
            localStorage.setItem('enjambre_token', result.token);
            localStorage.setItem('enjambre_user', userVal);

            ACCESS_TOKEN = result.token;
            USERNAME = userVal;

            showMainContent();
        } else {
            alert("Login Gagal. Periksa kembali akun Anda.");
        }
    } catch (err) {
        console.error("Login Error:", err);
```

*Gambar 17:Implementasi Event Listener pada Submit Login*



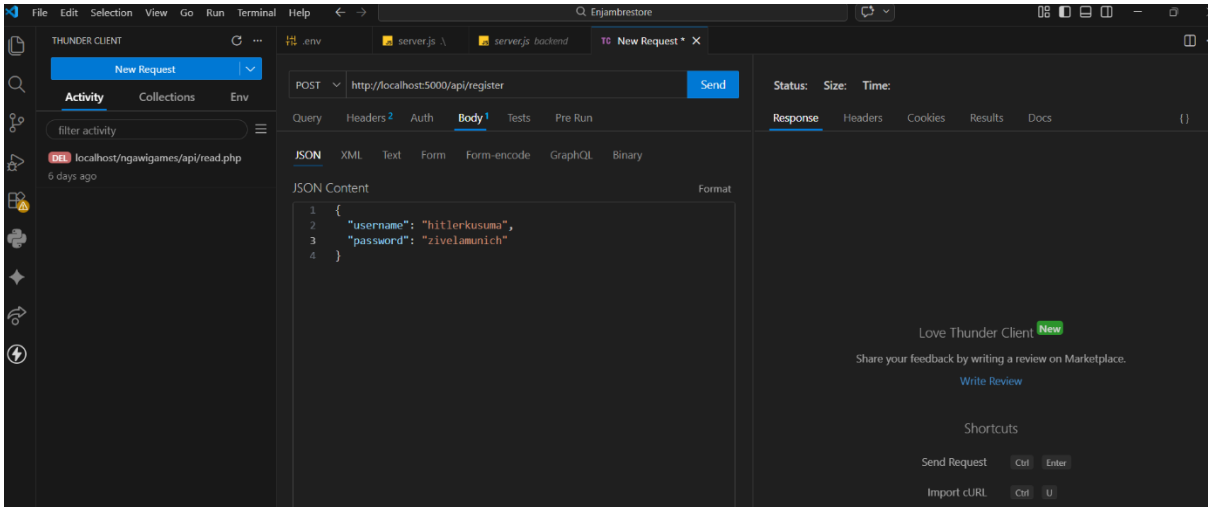

*Gambar 18:Aktivasi Live Server*




*Gambar 19:Tampilan Log In Awal*

## Konfigurasi Request POST untuk Autentikasi

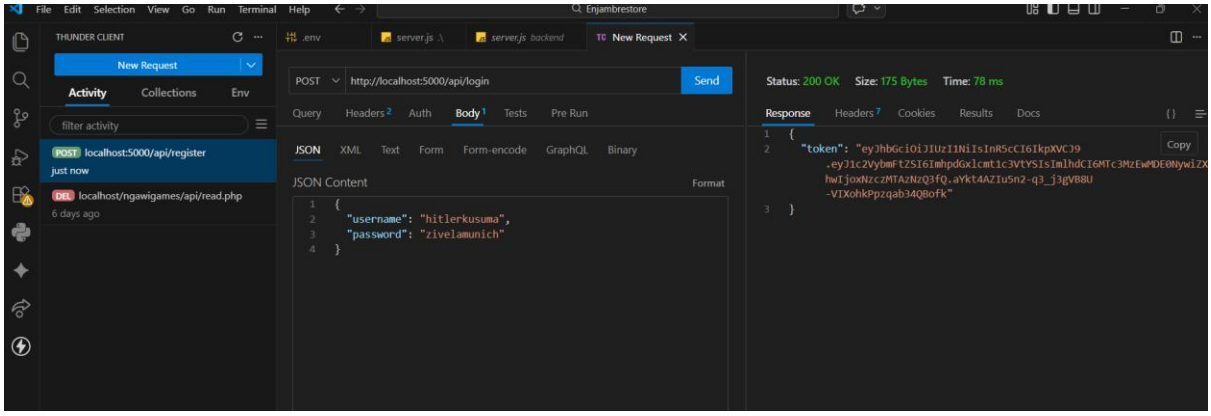
Metode fetch() dikonfigurasi menggunakan properti method: 'POST' untuk mengirimkan kredensial user ke *endpoint* /api/login. Data input dikonversi menjadi format JSON melalui JSON.stringify() untuk memenuhi standar pertukaran data pada *RESTful API*.




*Gambar 20:Konfigurasi Request POST untuk Autentikasi*

## Penanganan Response dan Ekstraksi Token

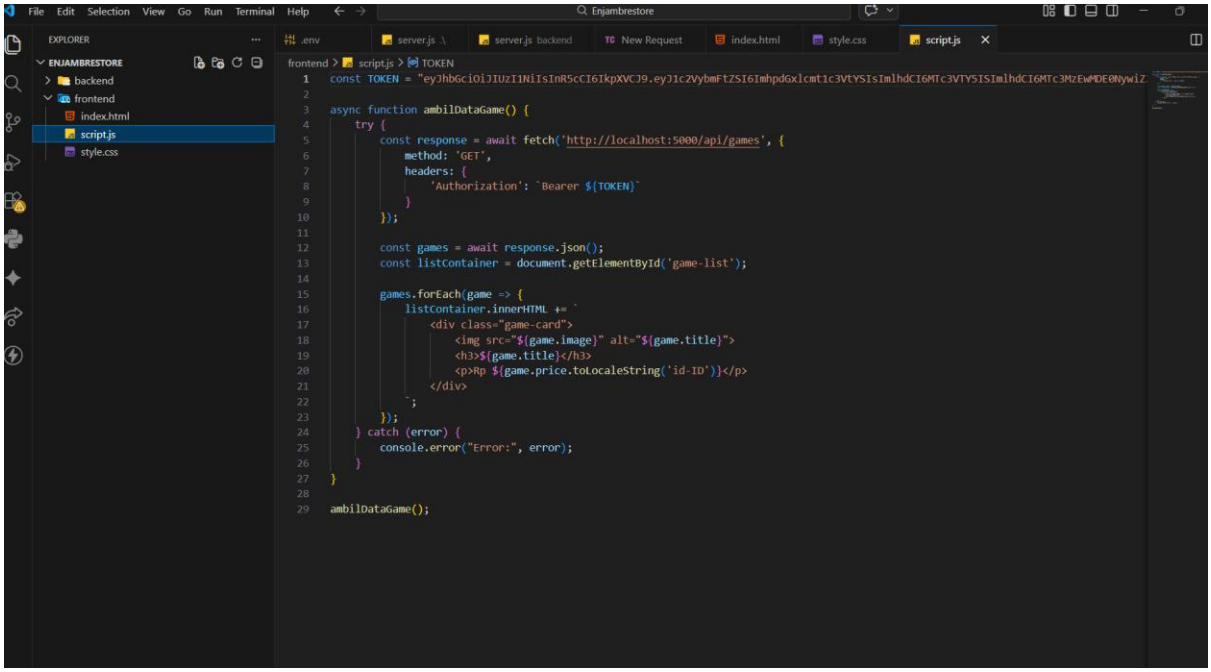
Sistem diatur untuk memproses respons dari server menggunakan .then(response => response.json()). Analisis dilakukan terhadap data kembalian untuk mendeteksi keberadaan *Access Token* yang menjadi bukti identitas digital pengguna yang sah.




*Gambar 21:Penanganan Response dan Ekstraksi Token*

## Penyimpanan Token pada LocalStorage

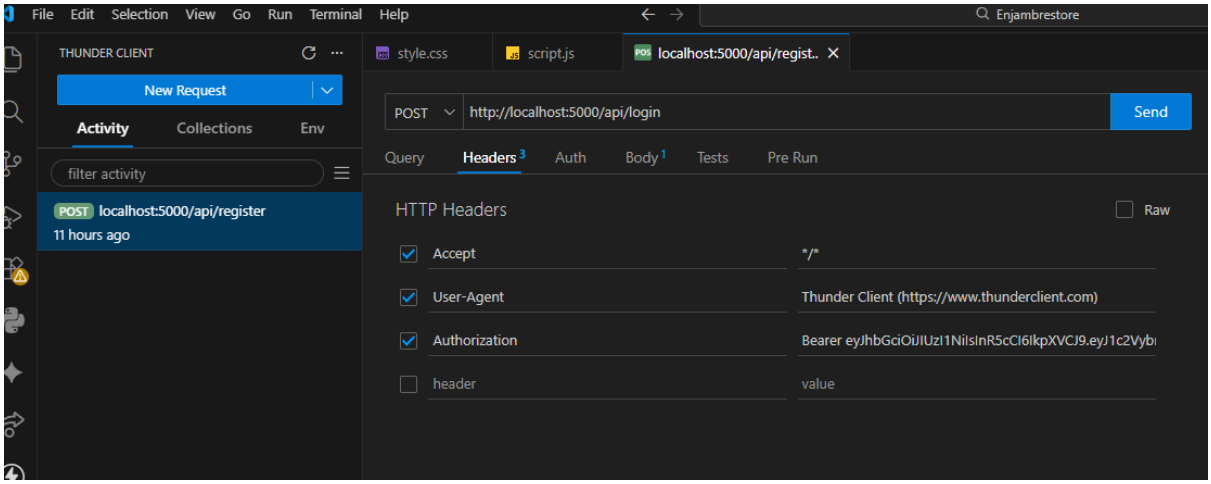
Setelah token berhasil didapatkan, fungsi localStorage.setItem('enjambre_token', token) dijalankan. Langkah ini krusial untuk menjaga status *session* pengguna agar tetap aktif selama melakukan navigasi antarhalaman tanpa perlu melakukan login ulang.




*Gambar 22:Penyimpanan Token pada LocalStorage*

### Implementasi Header Authorization Bearer

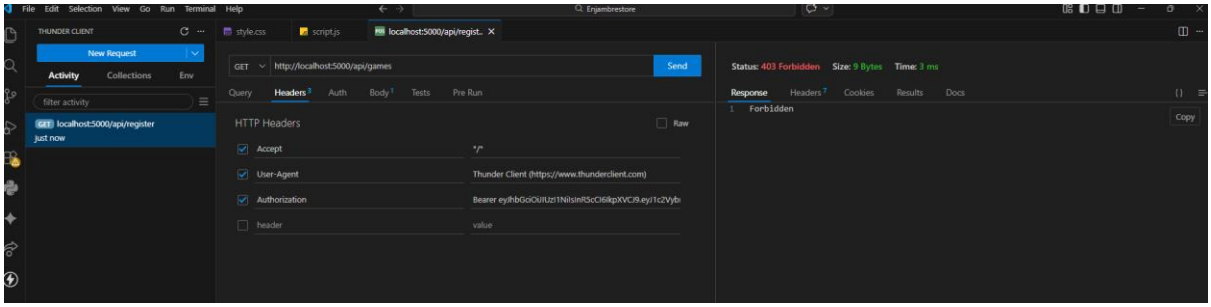
Setiap permintaan data selanjutnya (seperti daftar game) dikonfigurasi dengan menyertakan header 'Authorization': 'Bearer ' + token. Hal ini dilakukan untuk memenuhi protokol keamanan server yang mewajibkan validasi token pada setiap *endpoint* yang bersifat privat.




*Gambar 23:Implementasi Header Authorization Bearer*

### Pengambilan Data Produk dengan Request GET

Fungsi GET /api/games dieksekusi untuk menarik seluruh basis data produk game. Data hasil tarikan ini kemudian disimpan dalam variabel global atau *state* aplikasi sebelum diproses lebih lanjut oleh fungsi perenderan antarmuka.




*Gambar 24:Pengambilan Data Produk dengan Request GET*

## Penyusunan Logika Pembuatan Akun (Sign Up)

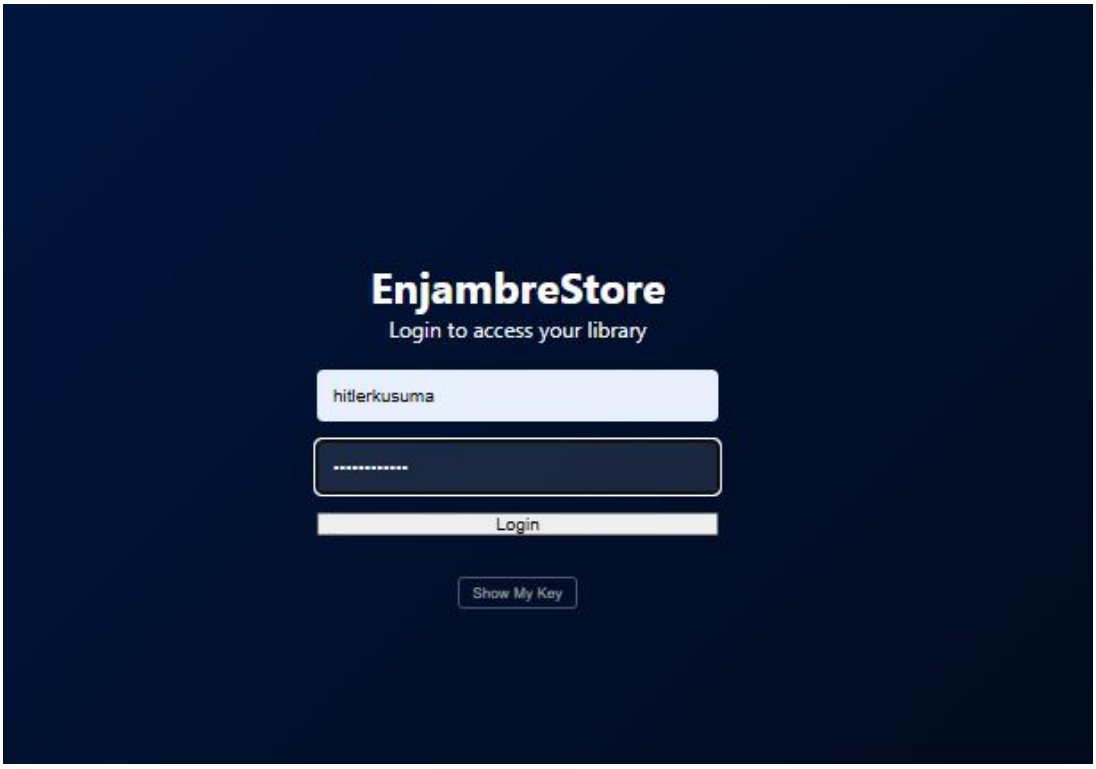
Mirip dengan sistem login, fitur registrasi diimplementasikan melalui metode POST ke *endpoint* /api/register. Skrip memastikan bahwa status kode respons 201 Created diterima dari server sebelum memberikan instruksi kepada user untuk melanjutkan ke tahap login.

```
e.css    script.js    server.js    localhost:5000/api/regist..
nd > server.js > games
const express = require('express');
const jwt = require('jsonwebtoken');
const bcrypt = require('bcryptjs');
const cors = require('cors');
require('dotenv').config();

const app = express();
app.use(express.json());
app.use(cors());

const hash = bcrypt.hashSync("zivelamunich", 10);
let users = [{ username: "hitlerkusuma", password: hash }];

let games = [
    { id: 1, title: "The Last of Us Part II",
        price: 800000, genre: "Action",
        image: "https://shared.fastly.steamstatic.com/store_item_assets/steam/apps/1888510/header.jpg",
        gameplayScreenshots: [
```

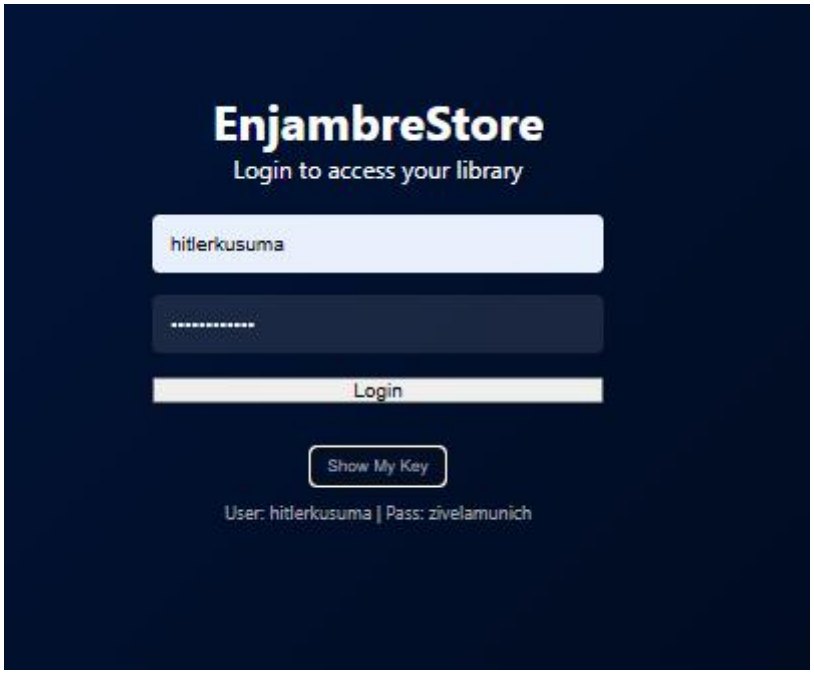
*Gambar 25:Penyusunan Logika Pembuatan Akun*

```
<body>
    <div id="login-screen" class="login-container">
        <h1>EnjambreStore</h1>
        <p>Login to access your library</p>
        <div class="form-group">
            <input type="text" id="username" placeholder="Username">
            <input type="password" id="password" placeholder="Password">
            <button onclick="handleLogin()">Login</button>
            <div class="simulation-hint">
                <button onclick="toggleKey()" class="btn-hint">Show My Key</button>
                <p id="key-info" style="display: none; margin-top: 10px; font-size: 12px; color: #aaa;">
                    User: hitlerkusuma | Pass: zivelamunich
                </p>
            </div>
        </div>
    </div>
```

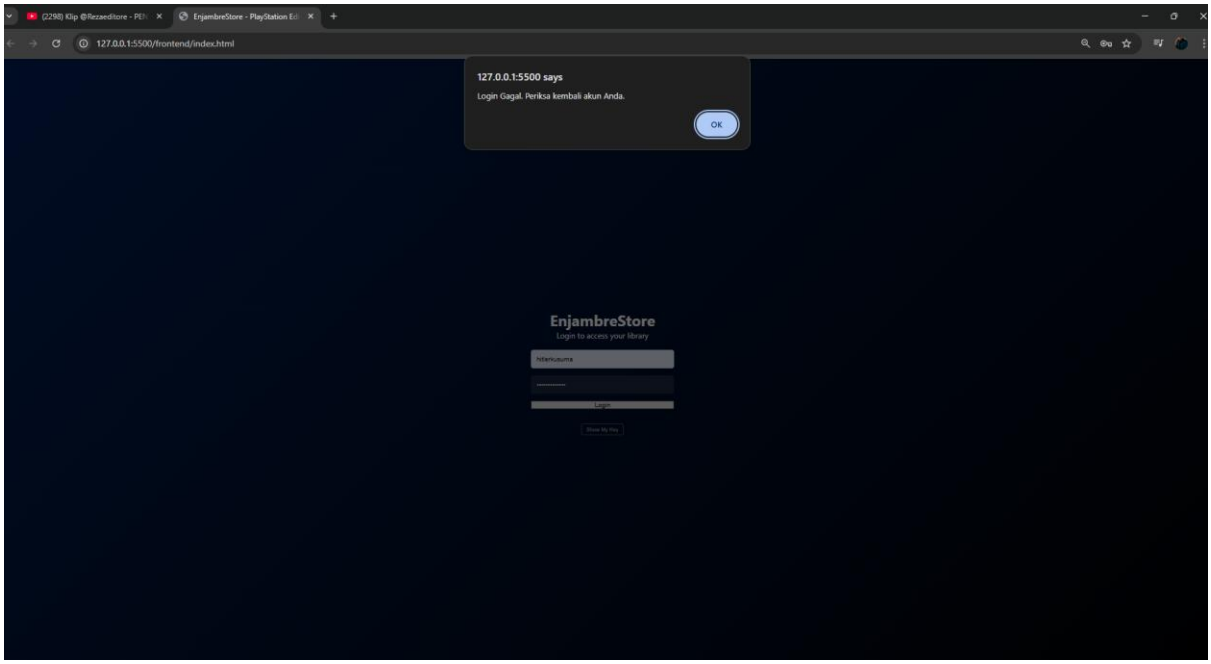
*Gambar 26:Kode Show Key*




*Gambar 27: Tampilan Log In*




*Gambar 28:Tampilan Show Key(Hanya dalam simulasi)*




*Gambar 29:Tampilan Salah/Error*

### Manajemen Session Nama Pengguna

Selain token, data profil dasar seperti username disimpan ke dalam localStorage menggunakan kunci enjambre_user. Data ini kemudian dipanggil kembali di halaman keranjang (cart.html) untuk memberikan personalisasi pada antarmuka pengguna.

```
        if (response.ok) {
            const result = await response.json();

            localStorage.setItem('isLoggedIn', 'true');
            localStorage.setItem('enjambre_token', result.token);
            localStorage.setItem('enjambre_user', userVal);

            ACCESS_TOKEN = result.token;
            USERNAME = userVal;

            showMainContent();
        } else {
            alert("Login Gagal. Periksa kembali akun Anda.");
        }
    } catch (err) {
        console.error("Login Error:", err);
        alert("Server tidak merespon.");
    }
}
```

*Gambar 30:Kode Manajemen Session Nama Pengguna*

**Fungsi Validasi Login Global**

Dibuat fungsi pengecekan otomatis pada setiap halaman yang memeriksa keberadaan enjambre_token. Jika token tidak ditemukan, sistem secara otomatis melakukan pengalihan lokasi (window.location.href) kembali ke halaman login sebagai bentuk proteksi akses terhadap konten utama.

```
document.addEventListener('DOMContentLoaded', () => {
    const isLoggedIn = localStorage.getItem('isLoggedIn');

    if (isLoggedIn === 'true' && ACCESS_TOKEN) {

        showMainContent();
    } else {

        showLoginScreen();
    }
    updateCartBadge();
});
```

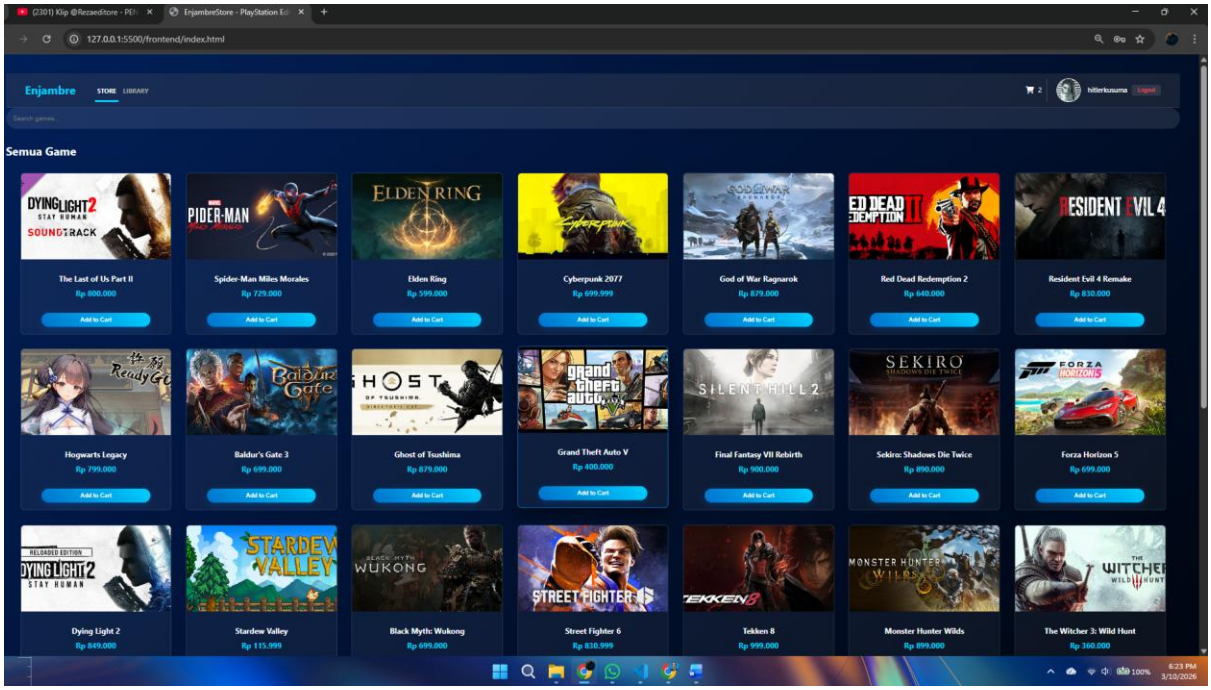
*Gambar 31:Fungsi Validasi Login Global*

## II.4. Hasil yang Didapat

Analisis terhadap hasil pengembangan "Enjambre Game Store" menunjukkan keberhasilan pemenuhan fungsionalitas aplikasi web yang *stateful*. Berikut adalah analisis mendalam terhadap komponen-komponen utama hasil praktikum:

### II.4.1 Analisis Sistem Autentikasi dan Proteksi Konten

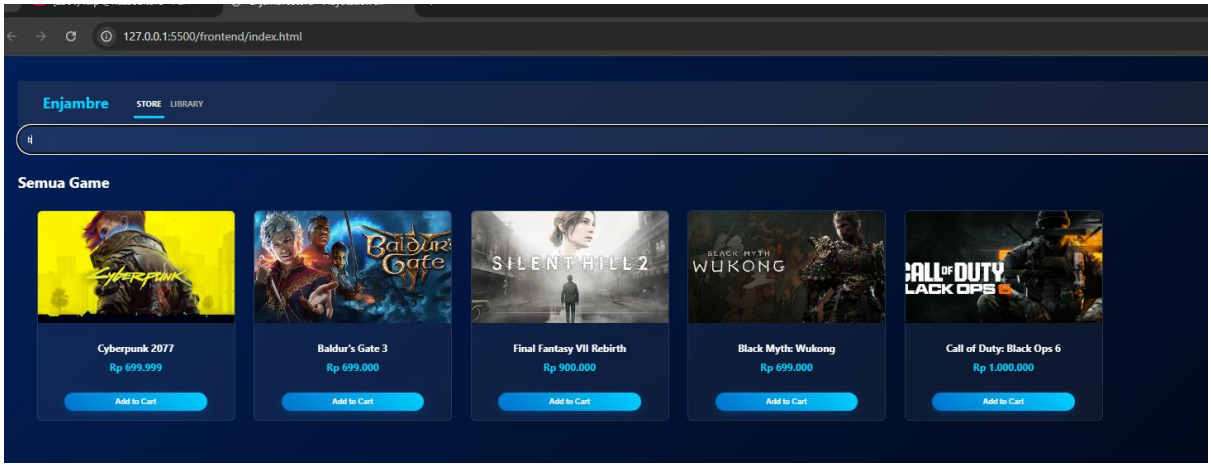
Sistem berhasil mengimplementasikan Mekanisme login berbasis token. Berdasarkan pengujian pada fungsi DOMContentLoaded, aplikasi mampu melakukan pengalihan layar (*screen switching*) secara instan antara login-screen dan main-content tanpa proses *refresh* halaman. Penggunaan localStorage untuk menyimpan enjambre_token dan isLoggedIn terbukti efektif dalam menjaga sesi pengguna, di mana identitas pengguna tetap tertampil pada elemen header secara akurat melalui manajemen *session* enjambre_user.




*Gambar 32:Tampilan Dashboard*

### II.4.2 Analisis Perenderan Produk Dinamis dan Filter

Hasil pengambilan data dari API berhasil dikonversi menjadi elemen DOM melalui fungsi renderGames(). Penggunaan metode toLocaleString('id-ID') menghasilkan format mata uang Rupiah yang presisi pada setiap kartu game. Fitur pencarian pada fungsi filterGames() mampu melakukan penyaringan data secara *real-time* dengan memanipulasi array STORE_DATA secara lokal, yang meningkatkan responsivitas antarmuka karena tidak memerlukan permintaan data ulang ke server untuk setiap kata kunci pencarian.

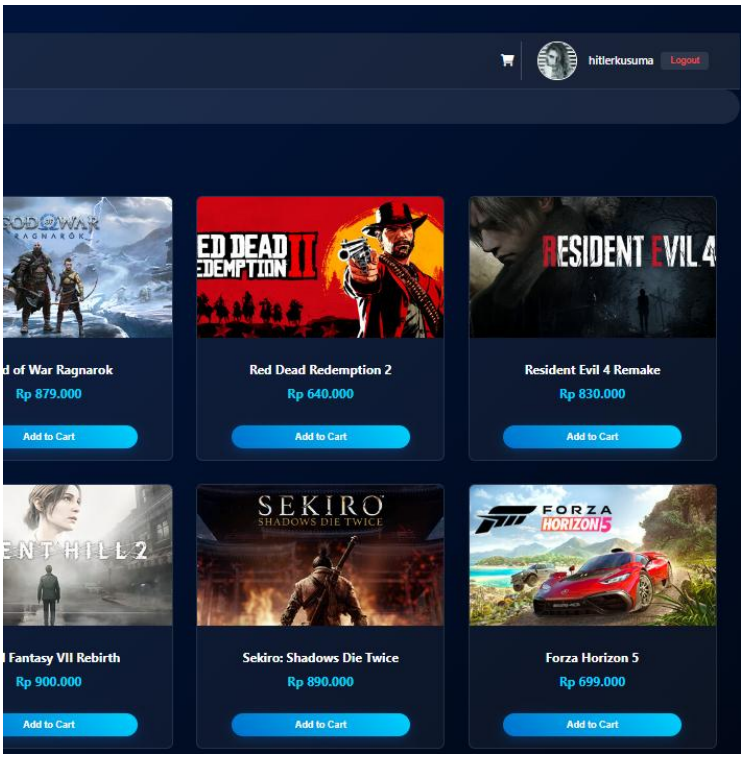



*Gambar 33:Tampilan Perenderan Produk Dinamis dan Filter*

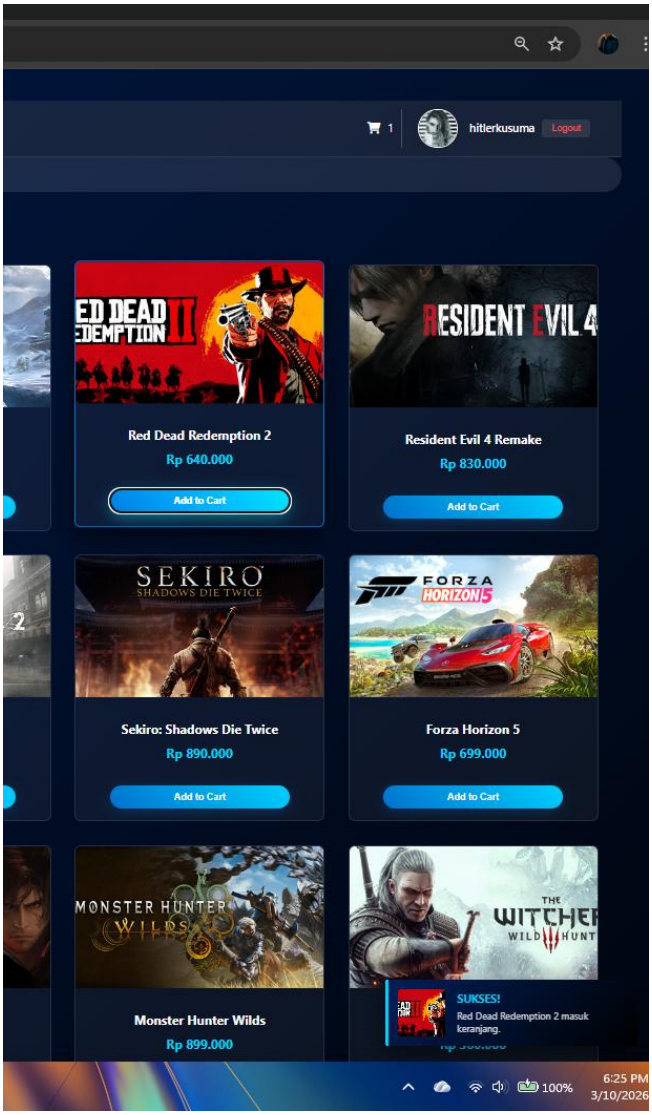
### II.4.3 Analisis Manajemen Keranjang dan Notifikasi

Fungsi handleAddToCart() berhasil melakukan validasi duplikasi data menggunakan metode .some(). Analisis terhadap objek cart di localStorage menunjukkan bahwa data disimpan dalam format *Array of Objects* yang konsisten. Keberhasilan penambahan item

ditandai dengan munculnya epic-notif yang secara dinamis mengambil properti image dan title dari objek game terkait. Perhitungan jumlah item pada cart-badge diperbarui secara otomatis melalui fungsi updateCartBadge() setiap kali terjadi perubahan pada memori lokal.



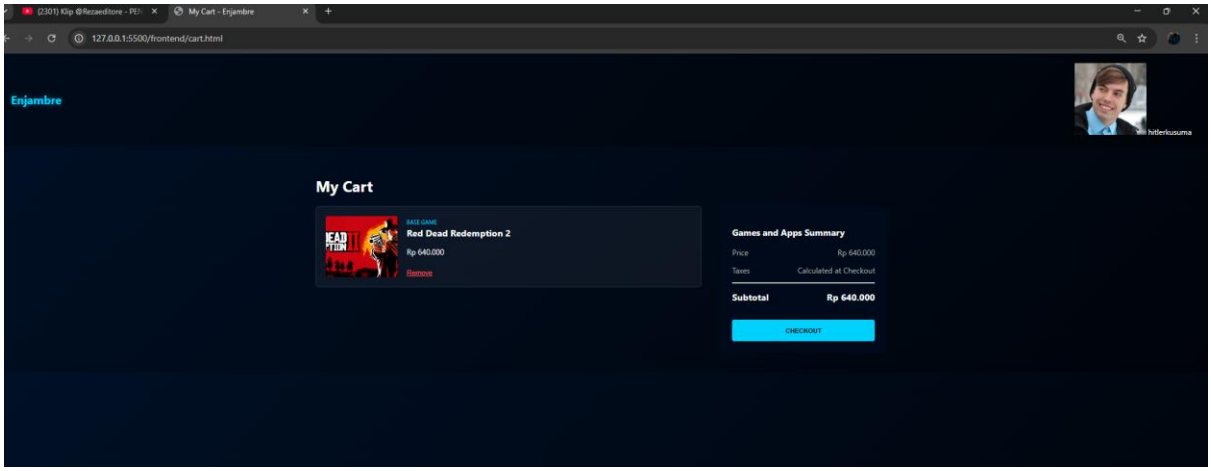

*Gambar 34:Tampilan Fungsionalitas icon keranjang*



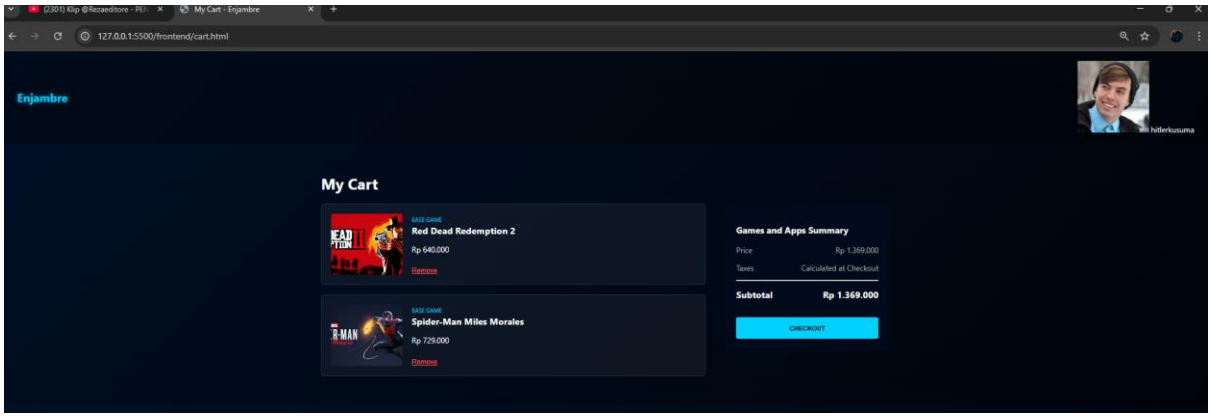

*Gambar 35:Tampilan data bertambah ke cart.html*

### II.4.4. Analisis Detail Produk dan Transaksi

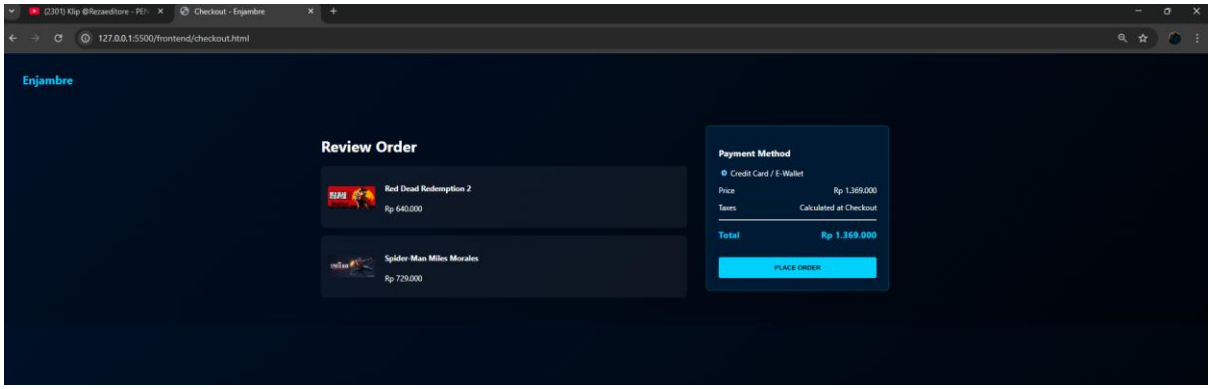
Fungsionalitas showGameDetail() memberikan visualisasi mendalam dengan memanfaatkan *template literals* untuk menginjeksi data spesifik game ke dalam elemen detail-content. Proses transisi dari keranjang menuju koleksi permanen dilakukan dengan pemindahan *state* data antara kunci enjambre_cart ke enjambre_library. Hasil pengujian menunjukkan bahwa setelah proses konfirmasi transaksi, memori keranjang dibersihkan secara total menggunakan removeItem(), dan data tersebut muncul sebagai koleksi baru pada halaman library.html.



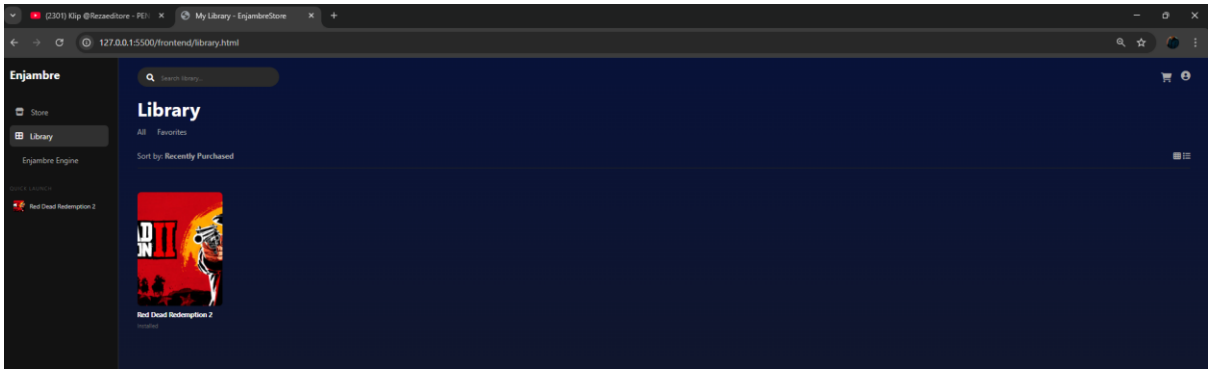

*Gambar 36:Tampilan Cart.html*



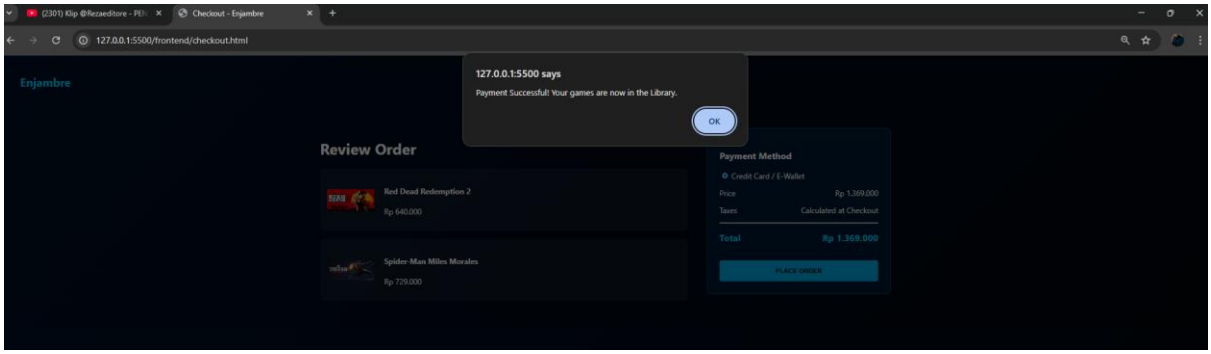

*Gambar 37:Tampilan Game Bertambah"*



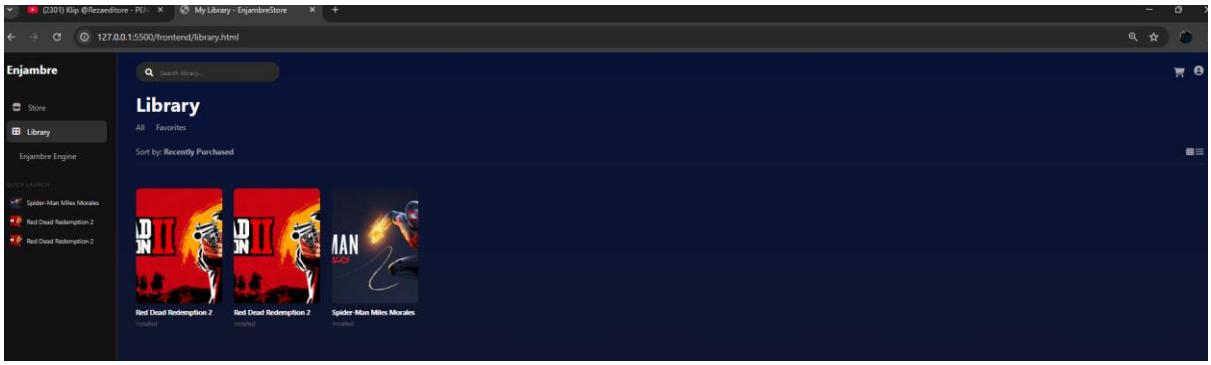

*Gambar 38:Tampilan Checkout.html*




*Gambar 39: Tampilan Library sebelum Checkout*




*Gambar 40:Notifikasi Pembelian Berhasil*



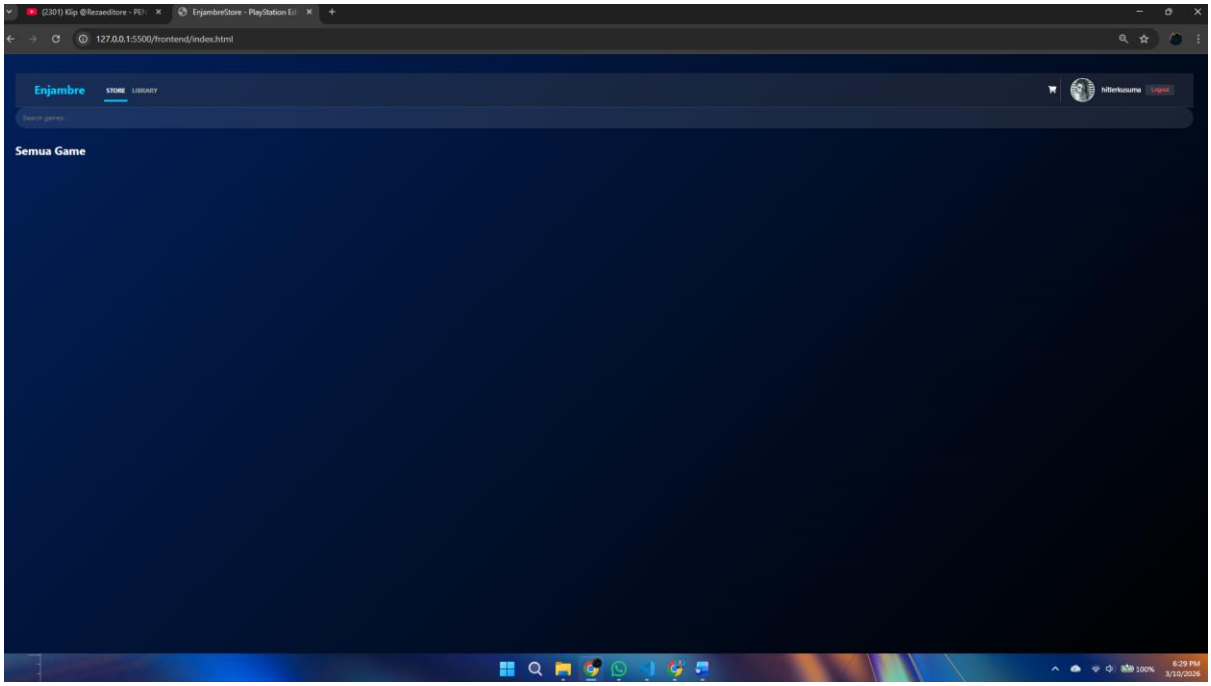

*Gambar 41:Tampilan Game Bertambah*

### II.4.5. Analisis Penanganan Error dan Stabilitas

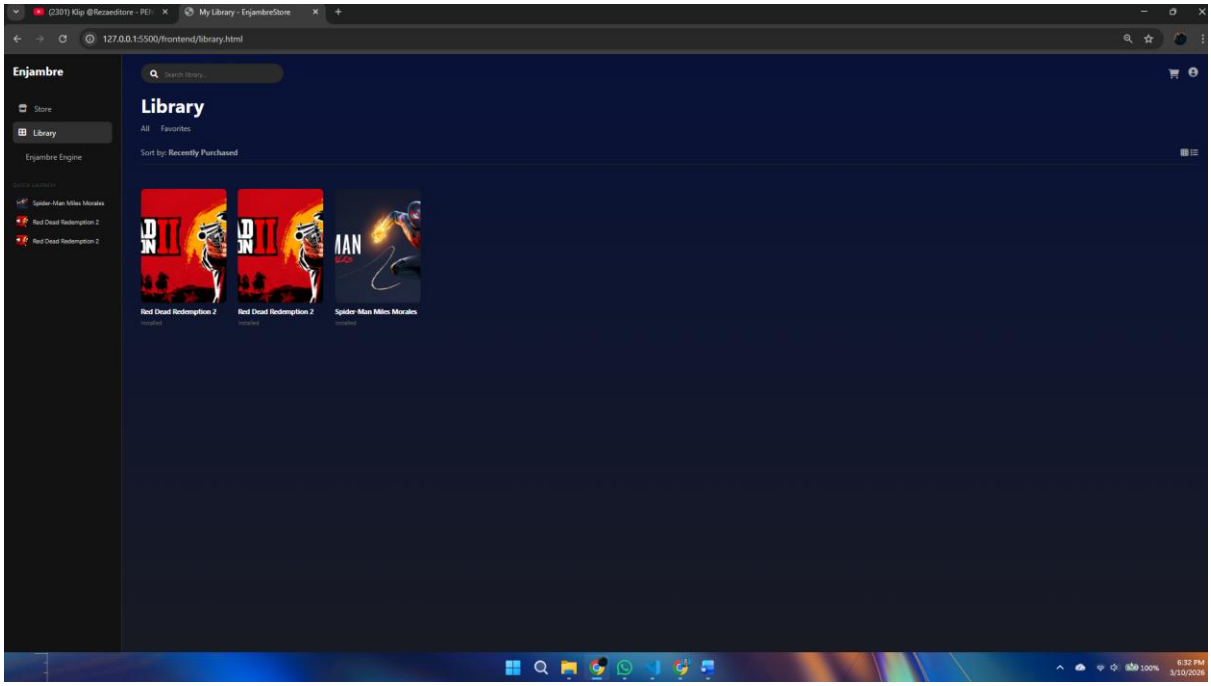
Selama fase pengujian, sistem menunjukkan stabilitas dalam menangani data kosong. Penggunaan operator OR (|| []) pada saat pengambilan data dari localStorage mencegah terjadinya error *null pointer*. Selain itu, penanganan error pada blok try-catch di fungsi fetch memastikan aplikasi tidak berhenti bekerja (*crash*) saat koneksi ke server lokal terputus, melainkan memberikan informasi yang tepat melalui konsol pengembang.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS Z:\Unsil-PROJEK\S4\Enjambrestore> cd backend
PS Z:\Unsil-PROJEK\S4\Enjambrestore\backend> ^C
PS Z:\Unsil-PROJEK\S4\Enjambrestore\backend> node server.js
[dotenv@17.3.1] injecting env (2) from .env -- tip: write to custom object with { processEnv: myObject }
Active on 5000
PS Z:\Unsil-PROJEK\S4\Enjambrestore\backend>
```

*Gambar 42:Server dimatikan/Stop*




*Gambar 43:Tampilan Dashboard Jika Server Lokal Mati*




*Gambar 44:Tampilan Library Saat Server Lokal Mati*

# BAB III

# PENUTUP

## III.I. Kesimpulan Praktikum

Berdasarkan seluruh tahapan pengembangan dan pengujian yang telah dilakukan, dapat disimpulkan bahwa:

1. **Integrasi Client-Side dan API**: Implementasi *Vanilla JavaScript* melalui metode *Asynchronous Fetch* berhasil menghubungkan antarmuka pengguna dengan data dinamis dari server secara stabil, terbukti dengan perenderan kartu game yang presisi dan responsif.
2. **Efektivitas LocalStorage**: Penggunaan *Web Storage API* sebagai instrumen manajemen status (*state management*) terbukti sangat efektif untuk menjaga persistensi data keranjang belanja dan koleksi *library* pengguna tanpa membebani kinerja server untuk tugas-tugas penyimpanan sementara.
3. **Logika Transaksi**: Alur operasional aplikasi dari tahap login, pemilihan produk, hingga simulasi *checkout* telah mengikuti standar logika *E-Commerce* yang benar, di mana data berpindah secara akurat dari memori keranjang ke memori kepemilikan permanen pengguna.
4. **Keandalan Sistem Proteksi**: Fungsi validasi login global yang diimplementasikan pada setiap halaman berhasil mencegah akses ilegal terhadap konten utama, memastikan hanya pengguna dengan token valid yang dapat berinteraksi dengan sistem.

## III.II. Analisis Dan Evaluasi

Meskipun fungsionalitas utama telah terpenuhi, terdapat beberapa poin hasil analisis teknis yang perlu dievaluasi untuk pengembangan lebih lanjut:

- **Keamanan Data pada Client-Side**: Analisis terhadap penggunaan *LocalStorage* menunjukkan adanya celah keamanan di mana pengguna yang memiliki pemahaman teknis dapat memanipulasi data keranjang atau koleksi melalui konsol peramban. Evaluasi menyarankan agar validasi akhir transaksi di masa mendatang harus dilakukan sepenuhnya di sisi *backend* untuk menjamin integritas data.
- **Penanganan Error 403 (Forbidden)**: Selama praktikum, ditemukan kondisi di mana permintaan data ditolak oleh server akibat token yang tidak valid atau kedaluwarsa. Hal ini menekankan pentingnya implementasi fitur *refresh token* atau penanganan error yang lebih informatif pada antarmuka pengguna agar tidak terjadi kebingungan saat akses ditolak.
- **Optimalisasi Pengalaman Pengguna (UX)**: Penggunaan notifikasi "Epic Notification" memberikan umpan balik visual yang baik, namun perlu adanya

penambahan fitur *loading state* (skeleton screen) saat proses pengambilan data dari API berlangsung untuk meningkatkan kenyamanan pengguna saat koneksi internet lambat.

- **Skalabilitas Kode**: Struktur kode yang saat ini masih bersifat prosedural di beberapa bagian dapat dioptimalkan kembali menggunakan pendekatan *Object-Oriented Programming* (OOP) atau komponen modular agar lebih mudah dipelihara (*maintainable*) jika fitur aplikasi bertambah di masa depan.

# DAFTAR PUSTAKA


Duckett, J. (2014). *JavaScript and JQuery: Interactive Front-End Web Development*. Wiley.

MDN Web Docs. (2024). *Using the Web Storage API*. Mozilla Foundation.

Haverbeke, M. (2018). *Eloquent JavaScript: A Modern Introduction to Programming*. No Starch Press.

Freeman, A. (2022). *Pro JavaScript Design Patterns*. Apress.

Flanagan, D. (2020). *JavaScript: The Definitive Guide*. O'Reilly Media.

Marcotte, E. (2011). *Responsive Web Design*. A Book Apart.

Gasston, P. (2013). *The Book of CSS3: A Developer's Guide to the Future of Web Design*. No Starch Press.

Zakas, N. C. (2016). *Understanding ECMAScript 6*. No Starch Press.

Simpson, K. (2015). *You Don't Know JS: Scope & Closures*. O'Reilly Media.

W3Schools. (2024). *JavaScript Web Storage API Reference*.